

密碼學簡介

建國中學 張晏誠

學習動機

一切都要從國中時說起，那時接觸到了遊戲“看門狗”，遊戲內容是扮演一名駭客在城市中利用駭客技術控制電子設備來完成任務，當時的我被遊戲中駭客神通廣大的技巧及化腐朽為神奇感到驚嘆，也因此想要學習先關資安技術，但當我開始閱讀相關書籍時這股熱潮馬上就消退了，當時的我第一次體驗到什麼叫做“每個字都看得懂，但連起來後沒一個字看得懂”。再接觸相關知識時已經是高二了，在高一升高二的暑假有一次逛誠品時，發現了一本名為“圖解密碼學”的書籍，讓我想起國中的那段小插曲，因此秉持著回味及好奇的角度去翻閱，結果竟一發不可收拾，儘管在閱讀時依舊有點吃力，但彷彿已經開始感受到密碼學中加解密及應用的奧妙，自從那時起，我會開始在課餘時間觀看密碼學相關書籍甚至是線上課，儘管學的還稱不上專業，卻能讓我感受到學習的樂趣。

很多人都說如果要增強自己對某事物的了解及知識，最好的方式就是利用“教學”的方式將自己現學的知識給講出來，因此我決定要將我所學的密碼學知識用簡介及講解的方式來呈現，除了可以幫助其他人拓展新知之外，也能加強並檢視我先前學所學的掌握度。

過程及待改進

在製作的過程中其實就是在反芻自己現有的知識，因此在思考過程跟比對資料時常常會發現自己有一些理解錯誤及先前沒有想過的問題，而解決這些問題正是我之所以製作這份簡報的目的，因此我也從中學習並修正了許多知識點。除了內容之外，我覺得我的這份簡報有一項需要改進的點是排版和呈現視覺體驗，因為我當初在做的時候只重內容，所以忽略了簡報整體的編排及整潔度，因此整份簡報看起來亂亂的不會勾起他人閱讀的慾望，希望未來在大學能去試著學習如何做版面的編排及簡報的整理。

未來展望

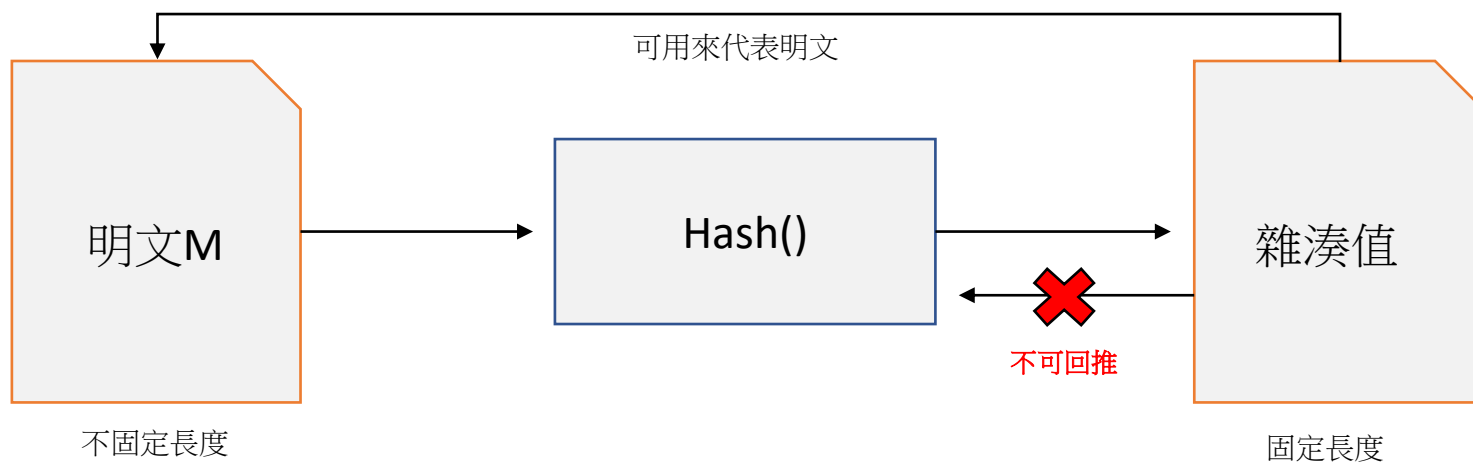
我希望未來能以密碼學知識當作我在資安領域的第一塊版圖並往其他版圖擴散，例如利用自學或大學的課程學習網頁安全、計算機安全或者開始學習一些硬體知識，讓我可以創造出一套結合軟硬體的資訊安全系統或是專案。最後，我希望在學習了資安相關知識後可以趕在大學畢業前參加好厲駭或者是 AIS3 來證明自己所學習的成果。

參考資料

- 圖解密碼學與比特幣原理 -- 結城浩 著
- 圖解密碼學與資訊安全 -- 余顯強 著
- Python 密碼學編程 -- Al Sweigart 著
- 數論與密碼學 -- Wilson Ren (Udemy 線上課)

雜湊

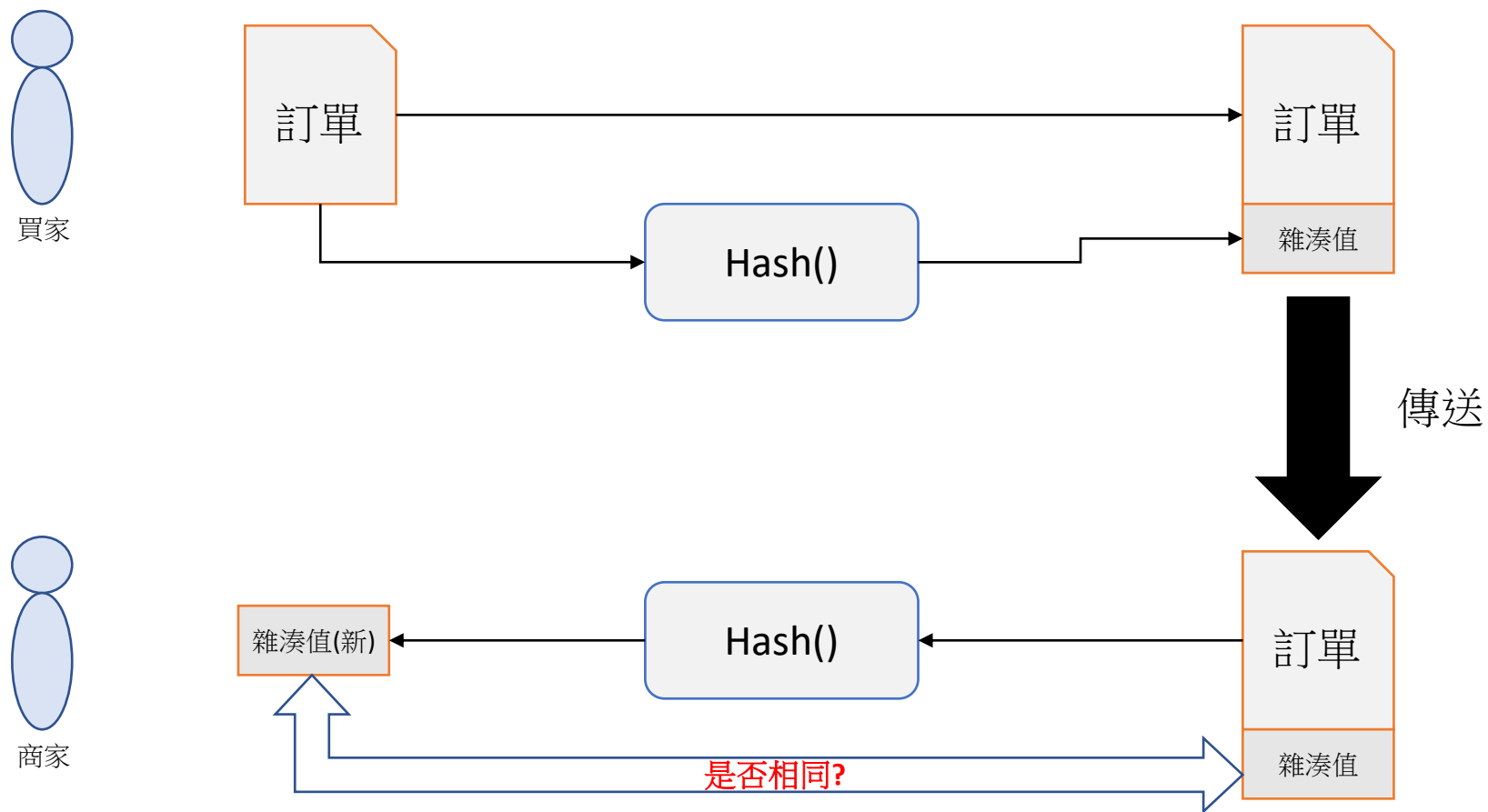
雜湊函數也稱作單向雜湊函數，主要的作用是對資料產出固定長度的摘要，且不同內容的資料經過單向雜湊函數計算後一定不會相同，即使資料只相差一位元(這裡其實不一定，如果雜湊碼太短可能導致有不同信息對到同個雜湊碼)。而名稱中的“單向”顧名思義就是無法再從雜湊後的亂碼回推當初的明文，但明文卻能輕易地經函數轉成雜湊碼，加上之前說的不同訊息對應不同雜湊碼，所以我們可以想像雜湊碼是我們文件的“身份碼”，經由對照即可知道這份文件是否經過修改及是否為所需文件。



理想的單向雜湊函數需要具備下列特性：

- (1) 函數具備確定性，及相同明文(M)會對應到相同雜湊值(D)
- (2) 輸入 M 能輕易地算出 D，但卻不能由 D 推回 M
- (3) 不存在M1 和 M2，其中 $M1 \neq M2$ 但 $D1 = D2$
- (4) 只要 M 的內容改變，D 一定隨之改變

基於上述的 (2) 和 (4)，我們可以得知雜湊值就像一個無法被冒充的辨識碼，且每一份訊息都有其“專屬”的雜湊碼，所以我們又稱雜湊碼為「數位指紋」，經由比對雜湊值來確認資料的完整性及是否遭到他人修改。如下面的範例所示，當買家送出訂單前會先把訂單雜湊後將雜湊碼附在後面方便商家檢查收到的訂單是否符合當初買家所發的訂單，而賣家收到訂單後的確認方式也非常簡單，將明文訂單雜湊後去比對是否與買家發的雜湊碼相同以確認內容是否與買家發的有誤。



雖然這樣看起來能確認資料的安全，但其實駭客只要傳送時攔截訊息並更改訊息再附上新的雜湊值，這樣商家就無法得知資料已被竄改並且還會因為雜湊值相同而傻傻地相信這份訂單就是出自買家手中，因此，實際的運作方式還需要更可靠的身分驗證機制，也就是加入非對稱式密碼處理的數位簽章。

MD5

首先介紹的是MD5，它的前身有MD、MD2 和 MD3，MD 後的數字代表的是版本號。MD5 跟前代比起來主要是加強了算法複雜度及不可逆性。MD5 可以輸入任一長度的資料，將之長度分割為 512 位元的訊息區段，經過執行產生四個 32 位元的資料，再將其組合而成一個 128 位元的雜湊值。

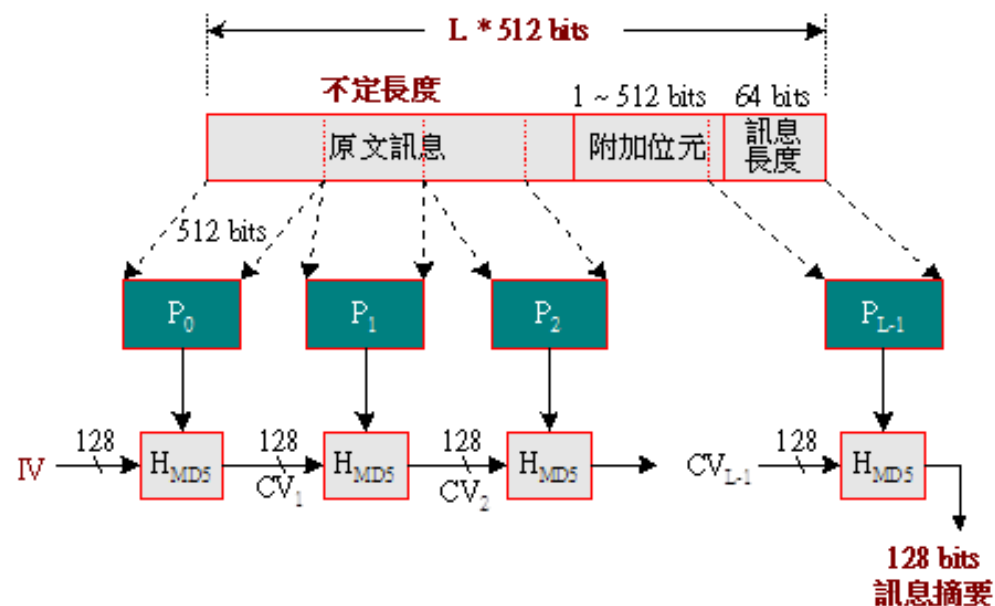
MD5處理流程：

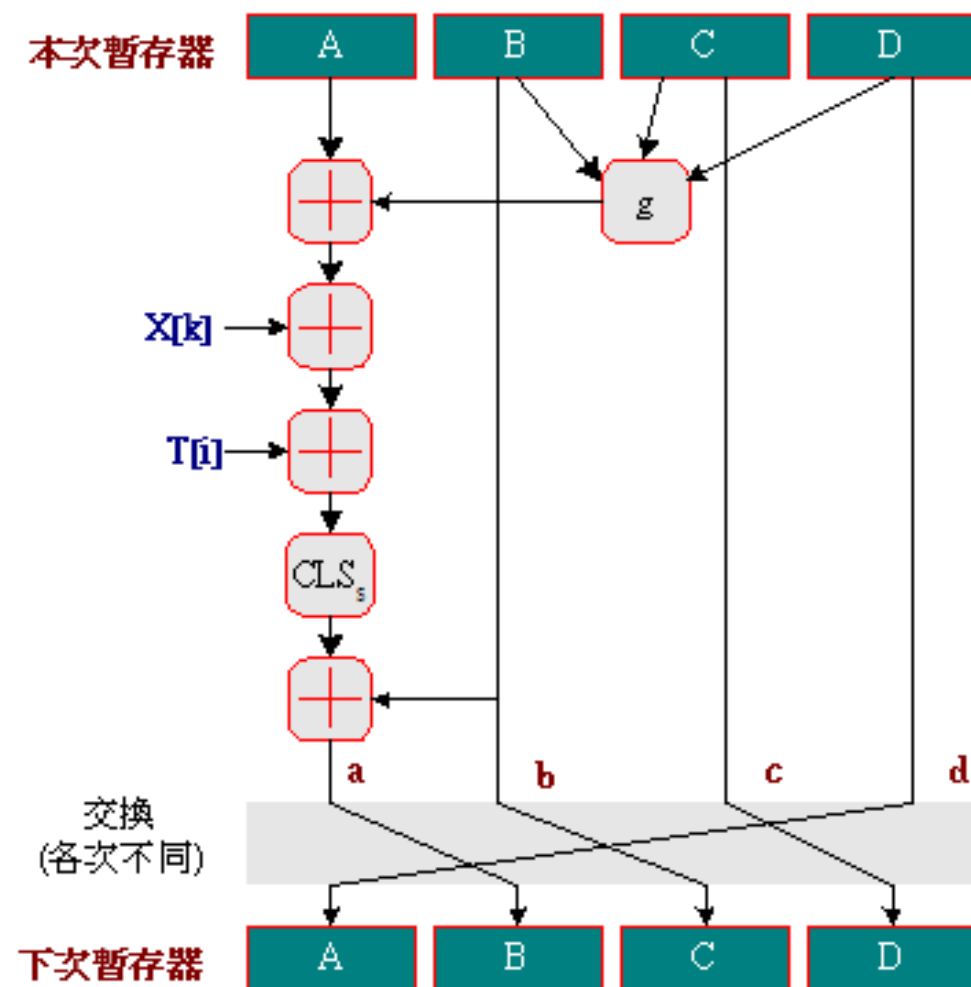
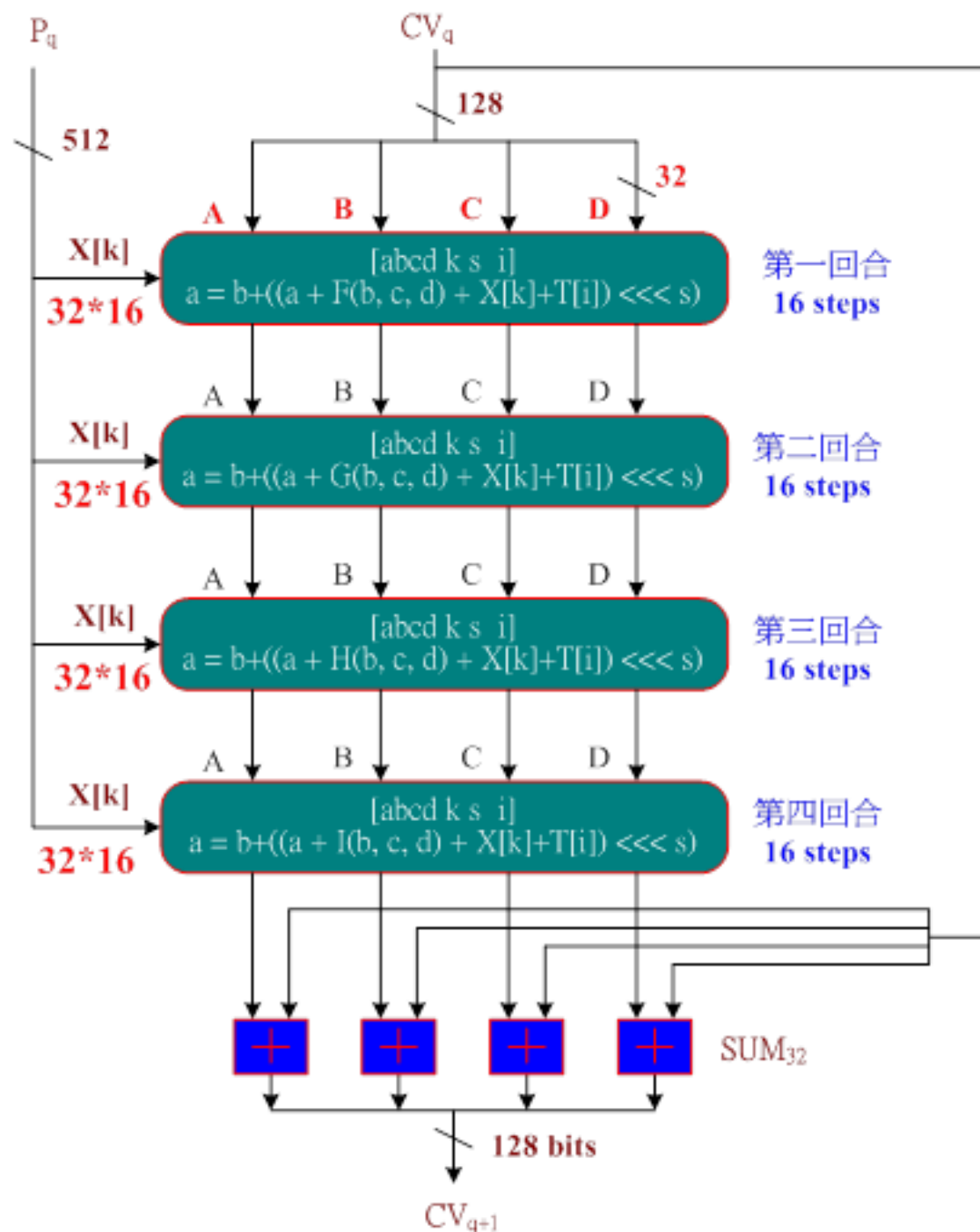
1. 附加位元：將訊息最後加上一些位元(一個 1 和 N 個 0)，使整個訊息長度 $\text{mod } 512 = 448(\text{位元})$ 。
2. 加上長度：將原始訊息的長度表示為 64 位元的資料，再將此長度資料附在訊息末端，如果原始訊息長度的資料大於 2 的 64 次方，就只讀取長度的最低 64 次方。
3. 設定緩衝區的初值：摘要的運算需要 4 個 32 位元的緩衝區來參與運算，其各有固定的初始值如下：
 $A = 01\ 23\ 45\ 67 \quad / \quad B = 89\ AB\ CD\ EF \quad / \quad C = FE\ DC\ BA\ 98 \quad / \quad D = 76\ 54\ 32\ 10$
4. 四輪迴圈運算：演算法主要的核心，由四個處理迴圈組成的壓縮模組，傳入要處理的訊息(512 bits) 及緩衝區 ABCD(共 128 bits)，而模組內的四個迴圈處理分別如下：

迴圈	函數	執行之邏輯
1	$F(b,c,d)$	$(b \wedge c) \vee (\neg b \wedge d)$
2	$G(b,c,d)$	$(b \wedge d) \vee (c \wedge \neg d)$
3	$H(b,c,d)$	$b \oplus c \oplus d$
4	$I(b,c,d)$	$c \oplus (b \vee \neg d)$

$\wedge$: AND $\vee$: OR $\oplus$: XOR $\neg$: NOT

5. 將步驟四的迴圈執行結果組合起來輸出固定長度為 **128** 位元的雜湊值，就完成訊息雜湊的產生。





影片輔助介紹

<https://youtu.be/Qsm7P9vVjeo>

更正：影片中我所說的位元加法其實是要取**32**位元的同餘加法(mod 2^{32})，為了確保每個位元加法後的值還是**32**位元。

SHA

SHA-1 是由美國國安局設計，且由美國國家標準與技術研究院公布為國家標準的雜湊演算法。SHA-1 是以 MD4 為基礎所發展出來的，所以其設計方式與 MD5 非常相似，差別在於兩者所產出的雜湊長度不同，SHA-1 的長度為 160 而 MD5 為 128 (位元)，更重要的是 SHA-1 比 MD5 更能有效防禦暴力破解的攻擊

SHA-1 廣泛的運用於網際網路傳輸的加密或是遠端連線的加密，如 SSL、TLS、和SSH，也可以用再加解密的 X.509 數位憑證等多種協議中。2017年，隨著網路技術及安全的顧慮，新一代的 SHA-2 成為了新的標準取代了原來的 SHA-1。SHA-2 正式發表於 2001 年，SHA-2 能輸出的雜湊值長度也從固定長度變成有多種長度可以選擇，其中的 SHA-256 是目前應用在數位簽章與數位憑證最廣泛的雜湊演算法，雖然仍是採用類似SHA-1 的演算法，但SHA-2 至今為止還無法有效地被破解。

算法和变体		输出散列 值长度 (bits)	中继散列 值长度 (bits)	数据区 块长度 (bits)	最大输入 消息长度 (bits)	循环 次数	使用到的运算符	碰撞攻 击 (bits)
MD5 (作为参考)		128	128 (4 × 32)	512	无限 ^[4]	64	And, Xor, Rot, Add (mod 2 ³²), Or	≤18 (发现碰撞)
SHA-0		160	160 (5 × 32)	512	2 ⁶⁴ - 1	80	And, Xor, Rot, Add (mod 2 ³²), Or	<34 (发现碰撞)
SHA-1		160	160 (5 × 32)	512	2 ⁶⁴ - 1	80		<63 ^[5] (发现碰撞 ^[6])
SHA-2	SHA-224 SHA-256	224 256	256 (8 × 32)	512	2 ⁶⁴ - 1	64	And, Xor, Rot, Add (mod 2 ³²), Or, Shr	112 128
	SHA-384 SHA-512 SHA-512/224 SHA-512/256	384 512 224 256	512 (8 × 64)	1024	2 ¹²⁸ - 1	80	And, Xor, Rot, Add (mod 2 ⁶⁴), Or, Shr	192 256 112 128
	SHA3-224 SHA3-256 SHA3-384 SHA3-512	224 256 384 512	1600 (5 × 5 × 64)	1152 1088 832 576	无限 ^[7]	24 ^[8]	And, Xor, Rot, Not	112 128 192 256
	SHAKE128 SHAKE256	d (arbitrary) d (arbitrary)		1344 1088				min(d/2, 128) min(d/2, 256)

表中能得知MD5 和 SHA-1 目前都已經可以被有效破解而變得不安全(雖然現在已經不會使用 MD5 做TLS/SSL 或 SSH 的加密及新文件的摘要，但還是能在一些軟體下載區及解壓縮檔案在確認完整性時看到它)，而且只能輸出固定長度的雜湊值。反觀SHA-2及 SHA-3 目前都還沒能發現有效的破解方式，而且輸出的雜湊值多元，其中最常使用的是SHA-256。(因為目前SHA-2 並沒有出現明顯的漏洞或安全性問題，所以 SHA-3 目前並未普及，主要是輔佐及預防未來 SHA-2 的安全性被破解)

SHA-2(以 SHA – 256 為例) 雜湊步驟：

跟 MD5 一樣，需要填充資料使訊息 MOD 512 = 448(位元)，在最後放入表資料長度的 64 位元，所以SHA - 2 能表示的資料長度一樣小於 2 的 64 次方。

1. 在運算前先來認識等等會用到的 F 函數

4.1.2 SHA-224 and SHA-256 Functions

SHA-224 and SHA-256 both use six logical functions, where *each function operates on 32-bit words*, which are represented as x , y , and z . The result of each function is a new 32-bit word.

$$Ch(x, y, z) = (x \wedge y) \oplus (\neg x \wedge z) \quad (4.2)$$

$$Maj(x, y, z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \quad (4.3)$$

$$\sum_0^{\{256\}}(x) = ROTR^2(x) \oplus ROTR^{13}(x) \oplus ROTR^{22}(x) \quad (4.4)$$

$$\sum_1^{\{256\}}(x) = ROTR^6(x) \oplus ROTR^{11}(x) \oplus ROTR^{25}(x) \quad (4.5)$$

$$\sigma_0^{\{256\}}(x) = ROTR^7(x) \oplus ROTR^{18}(x) \oplus SHR^3(x) \quad (4.6)$$

$$\sigma_1^{\{256\}}(x) = ROTR^{17}(x) \oplus ROTR^{19}(x) \oplus SHR^{10}(x) \quad (4.7)$$

3. 接著需要先初始化演算法中 8 個緩衝區塊：

5.3.3 SHA-256

For SHA-256, the initial hash value, $H^{(0)}$, shall consist of the following eight 32-bit words, in hex:

$$H_0^{(0)} = 6a09e667$$

$$H_1^{(0)} = bb67ae85$$

$$H_2^{(0)} = 3c6ef372$$

$$H_3^{(0)} = a54ff53a$$

$$H_4^{(0)} = 510e527f$$

$$H_5^{(0)} = 9b05688c$$

$$H_6^{(0)} = 1f83d9ab$$

$$H_7^{(0)} = 5be0cd19$$

These words were obtained by taking the first thirty-two bits of the fractional parts of the square roots of the first eight prime numbers.

4. W 處理 : SHA – 2 並非直接地將每份 32 bits 的原始訊息直接丟入做運算，而是會先經過 W 函數後才參與雜湊演算

1. Prepare the message schedule, $\{W_t\}$:

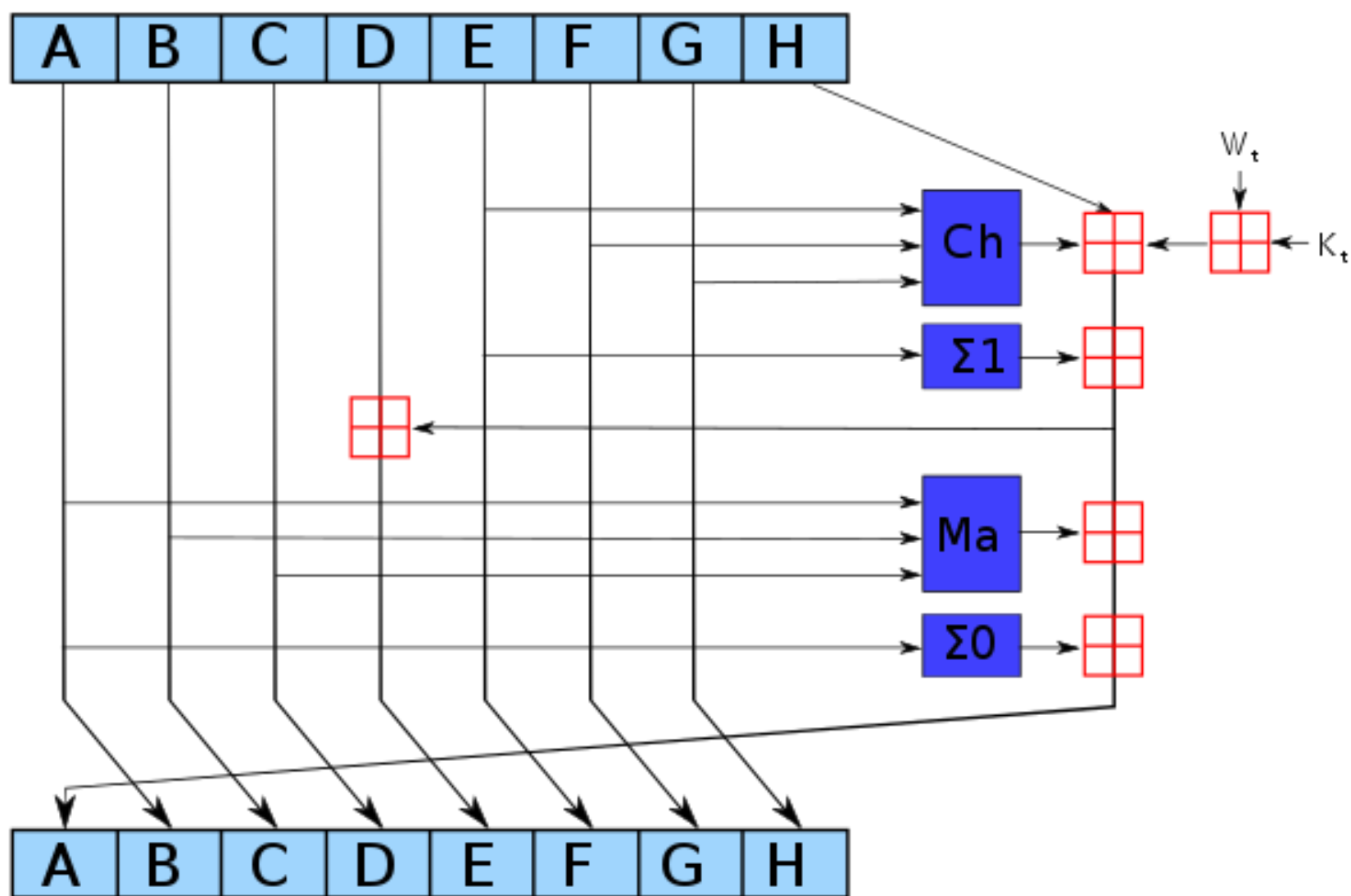
$$W_t = \begin{cases} M_t^{(i)} & 0 \leq t \leq 15 \\ \sigma_1^{\{256\}}(W_{t-2}) + W_{t-7} + \sigma_0^{\{256\}}(W_{t-15}) + W_{t-16} & 16 \leq t \leq 63 \end{cases}$$

4.2.2 SHA-224 and SHA-256 Constants

SHA-224 and SHA-256 use the same sequence of sixty-four constant 32-bit words, $K_0^{\{256\}}, K_1^{\{256\}}, \dots, K_{63}^{\{256\}}$. These words represent the first thirty-two bits of the fractional parts of the cube roots of the first sixty-four prime numbers. In hex, these constant words are (from left to right)

```
428a2f98 71374491 b5c0fbcf e9b5dba5 3956c25b 59f111f1 923f82a4 ab1c5ed5
d807aa98 12835b01 243185be 550c7dc3 72be5d74 80deb1fe 9bdc06a7 c19bf174
e49b69c1 efbe4786 0fc19dc6 240ca1cc 2de92c6f 4a7484aa 5cb0a9dc 76f988da
983e5152 a831c66d b00327c8 bf597fc7 c6e00bf3 d5a79147 06ca6351 14292967
27b70a85 2e1b2138 4d2c6dfc 53380d13 650a7354 766a0abb 81c2c92e 92722c85
a2bfe8a1 a81a664b c24b8b70 c76c51a3 d192e819 d6990624 f40e3585 106aa070
19a4c116 1e376c08 2748774c 34b0bcb5 391c0cb3 4ed8aa4a 5b9cca4f 682e6ff3
748f82ee 78a5636f 84c87814 8cc70208 90befffa a4506ceb bef9a3f7 c67178f2
```

← 等等會參與運算的 K



SHA 算法講解：<https://youtu.be/C3xZXQ9qDS4>

對稱式加密

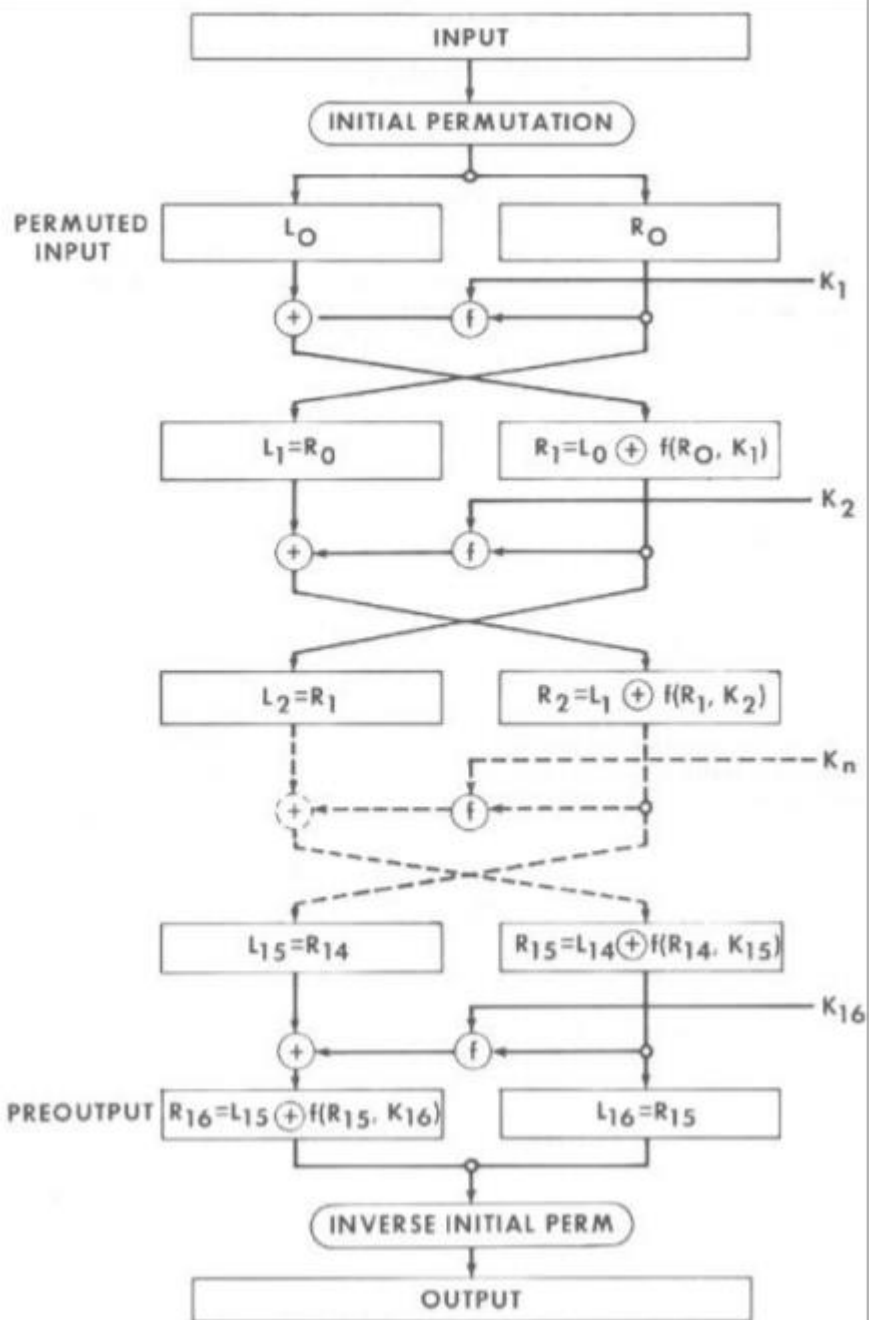
在介紹完雜湊後，接著要來介紹的是對稱式加密和非對稱式加密系統，這兩者與雜湊最大的差別式並非將資料純粹的打亂，使其無法被還原成明文，而是要能經由特定的金鑰將加密後的亂碼“完整”的還原成加密前的樣子，也就是明文。這裡我們首先介紹的是對稱式加密，其中的對稱指的是加密與解密用的是同一把金鑰，而其中加解密的運算其實跟雜湊的運算很像，都是利用位元的位移和互斥或(XOR)等利於電腦操作的位元運算。因此，對稱式加密的特色之一就是能有效的加解密大型文件，順道一提，其實在對稱式加密及非對稱式加密甚至是雜湊之間不會互相比較誰的安全性更好，因為他們各自有各自的“特色”，而這些“特色”在之後介紹的一些混和式加密或憑證時都會成為其中重要的“功能”，所以不存在“非對稱加密系統比對稱式系統安全，必須全體引用非對稱式加密法”這種說法，而是要正確的情況使用正確的加密方式才是上上策。

DES

DES (Data Encryption Standard) 是 IBM 公司於 1970 年代中期所發展出來的加密系統1997，年美國聯邦資訊處理標準 (FIPS) 採用的對稱式密碼，也被當時全球政府及銀行等機關廣泛使用，但隨著電腦功能的進步，DES現在已經可以在短時間內被暴力破解了，因此DES現在的功能幾乎只剩解密舊檔案了，但它依舊是推動對稱式加密技術重要的一環。

DES的運算系統大致上承襲了Feistel架構，因此有以下特性：

1. 金鑰長度：64 bits (56 bits 為主金鑰並實際參與計算，剩下 8 bits 用於奇偶校驗，不實際參與運算)
2. 區塊長度：在 DES 演算法中，明文與密文區塊都是 64 bits
3. 每個獨立區塊都經加密器重複運算 16 次，每次運算都使用一把獨立的子金鑰
4. 子鑰匙產生器：主鑰匙經過子鑰匙產生器後，產生 16 把獨立的子金鑰，每把子金鑰長 48 bits
5. 演算法相同：基本上加解密的演算法是相同的，因此解密就是將子金鑰以相反於加密時的順序輸入即可

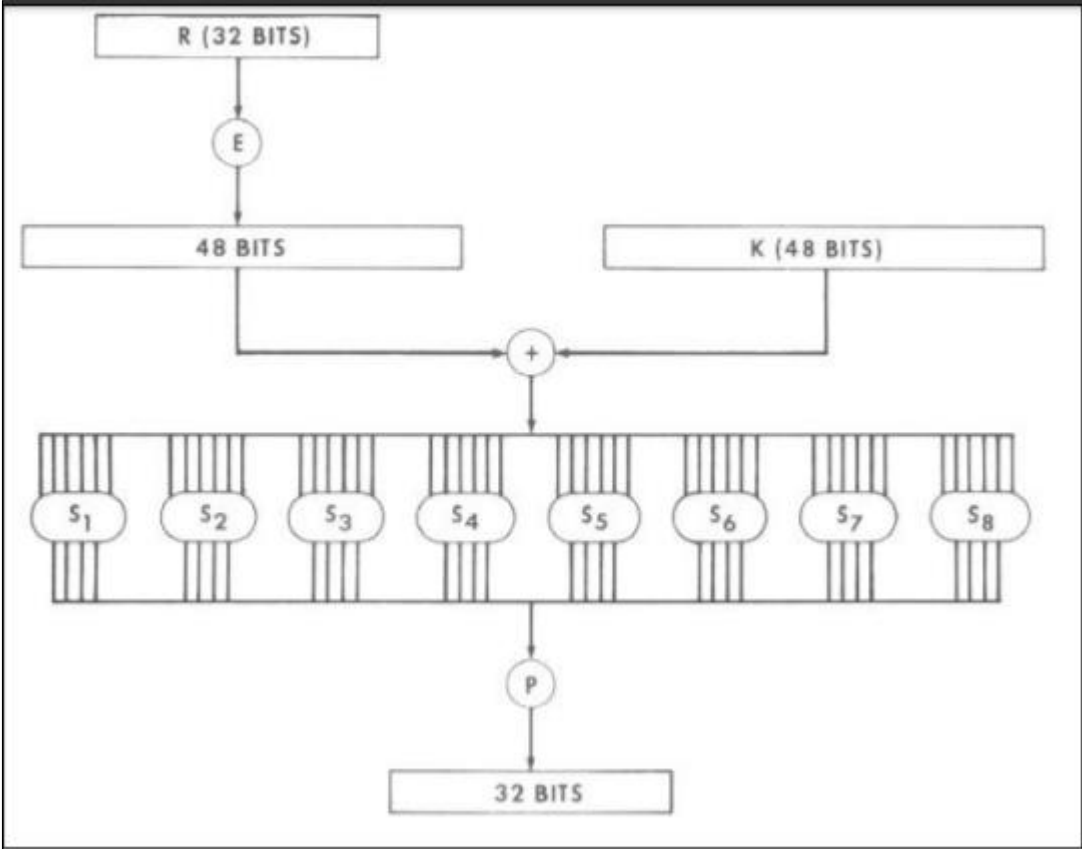


這裡先看一張大致的流程圖，簡單看一下步驟：

1. 先將資料做IP位移
2. 將資料切成左右兩部分
3. 資料經過 F 函數和 XOR 運算後左右互換
4. 將步驟 3 重複做 16 次
5. 將最後的資料做逆 IP 位移

The 64 bits of the input block to be enciphered are first subjected to the following permutation, called the initial permutation *IP*:

<u><i>IP</i></u>							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7



F函數 ↑

- 1. 擴張資料
- 2. 與回合金鑰混和
- 3. 進到 S 函數中運算，將 48 bits 轉為 32 bits 資料
- 4. 做 P 位移

<u><i>E</i> BIT-SELECTION TABLE</u>						<u><i>P</i></u>			
32	1	2	3	4	5	16	7	20	21
4	5	6	7	8	9	29	12	28	17
8	9	10	11	12	13	1	15	23	26
12	13	14	15	16	17	5	18	31	10
16	17	18	19	20	21	2	8	24	14
20	21	22	23	24	25	32	27	3	9
24	25	26	27	28	29	19	13	30	6
28	29	30	31	32	1	22	11	4	25

Column Number

Row

No.	<u>0</u>	<u>1</u>	<u>2</u>	<u>3</u>	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	<u>9</u>	<u>10</u>	<u>11</u>	<u>12</u>	<u>13</u>	<u>14</u>	<u>15</u>
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

金鑰的擴充與生成

1. 將金鑰做 PC-1 後，分成兩部分
2. 將金鑰做位移，輸出至 PC-2
3. PC-2的輸出值即為子金鑰，重複做 16 次步驟 2.

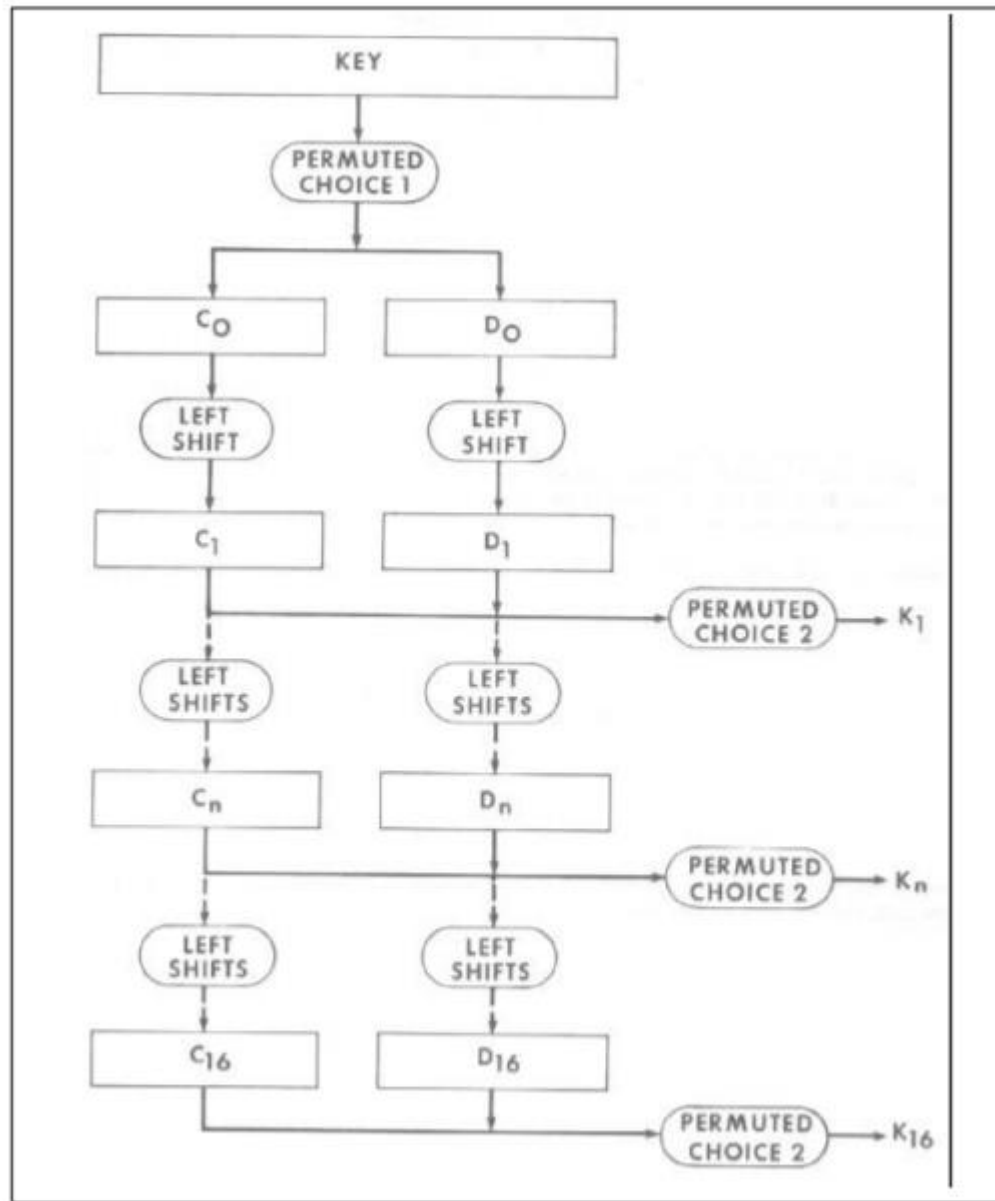


Figure 3. Key schedule calculation

PC-1

57	49	41	33	25	17	9
1	58	50	42	34	26	18
10	2	59	51	43	35	27
19	11	3	60	52	44	36
63	55	47	39	31	23	15
7	62	54	46	38	30	22
14	6	61	53	45	37	29
21	13	5	28	20	12	4

Iteration Number	Number of Left Shifts	<u>PC-2</u>					
1	1	14	17	11	24	1	5
2	1	3	28	15	6	21	10
3	2	23	19	12	4	26	8
4	2	16	7	27	20	13	2
5	2	41	52	31	37	47	55
6	2	30	40	51	45	33	48
7	2	44	49	39	56	34	53
8	2	46	42	50	36	29	32
9	1						
10	2						
11	2						
12	2						
13	2						
14	2						
15	2						
16	1						

DES 算法講解 : https://youtu.be/GoSjrd9ZU_M

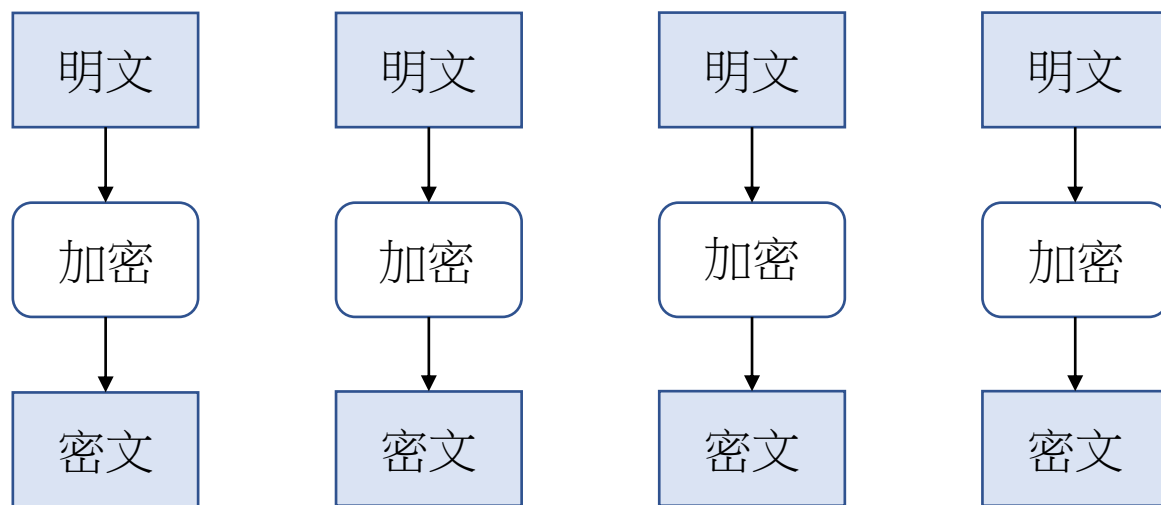
加密模式

在認識了對稱式加密之後，接著要認識的是加密的模式。例如我們熟知的對稱式加密**DES**，一次加密只能取明文中的**64bits** 加密，所以將明文分割並分別加密是必要的，但如果只是單純地分別加密在將密文連接起來很容易有安全疑慮，因為密文與明文有直接的順序關係，如果事先知道明文的內容就有機會可以利用變換密文的順序來修改明文的內容，過程中甚至不需要知道密鑰的值。因此，加密模式就是為了因應這種情形而設計出來的，利用**XOR**運算連接各個加密塊或使用**Feedback**模式複雜化加密過程都是不錯的方式，不同的模式都有各自的優缺點，能因應各種情形。接著會介紹**5**種不同的加密模式。

1. ECB (Electronic Code Book Mode)
2. CBC (Cipher Block Chaining Mode)
3. CFB (Cipher Feedback Mode)
4. OFB (Output Feedback Mode)
5. CTR (Counter Mode)

Electronic Code Book mode(ECB)

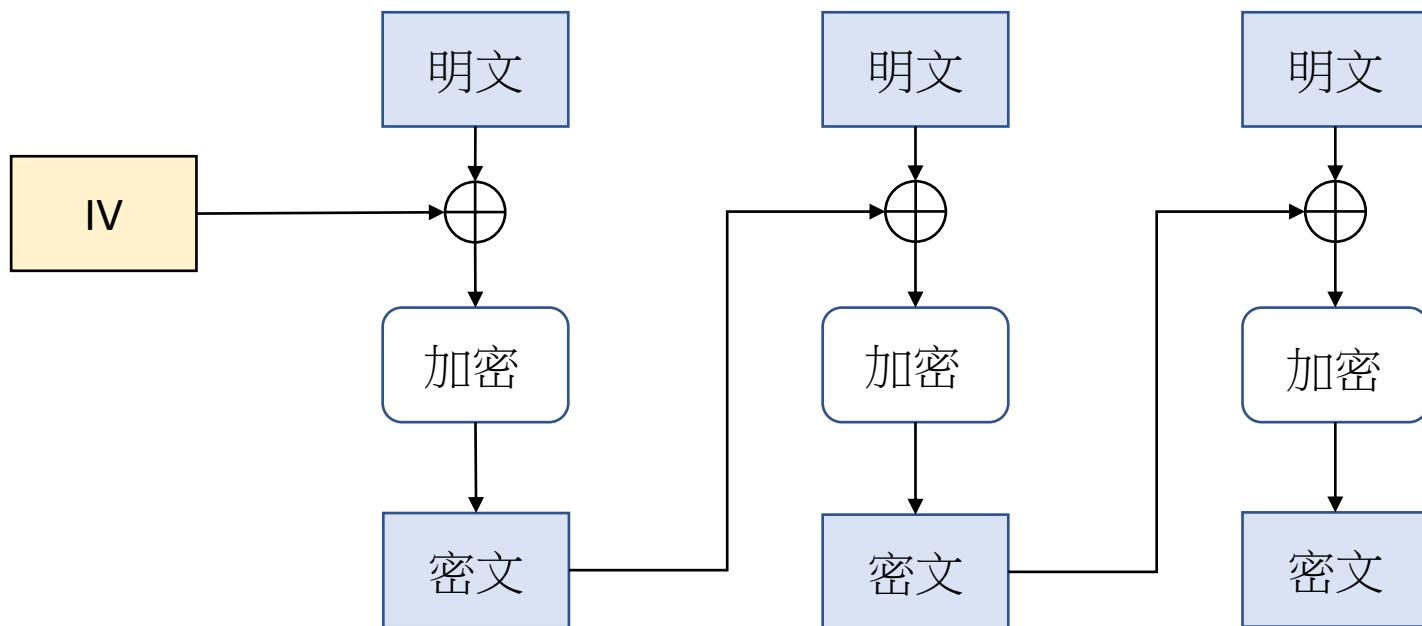
又稱電子密碼本模式，這是最危險但直觀的加密模式，簡單粗暴，以DES來說就是將明文直接以 64 bits 為單位直接加密，不經過任何處理直接將個別加密完的密文直接組在一起形成新密文(如圖1)，優點不是沒有，就是在短明文的情況下能簡單直觀的把明文加密，而且加密跟解密時每個區塊都能同步進行。而缺點就相對多而淺而易見，由於加密過程中沒有任何額外處理導致兩個大問題，第一點是因為明文與密文有直接的順序關係，所以攻擊者能以便能以調換密文順序來控制明文，過程中甚至不需要知道加密金鑰也能完成。第二點是同樣的明文中重複的部分在ECB中有可能會被加密成相同的密文進而增加風險，而這也是為什麼ECB只適用在短明文的情況。



(圖1)

CBC (Cipher Block Chain mode)

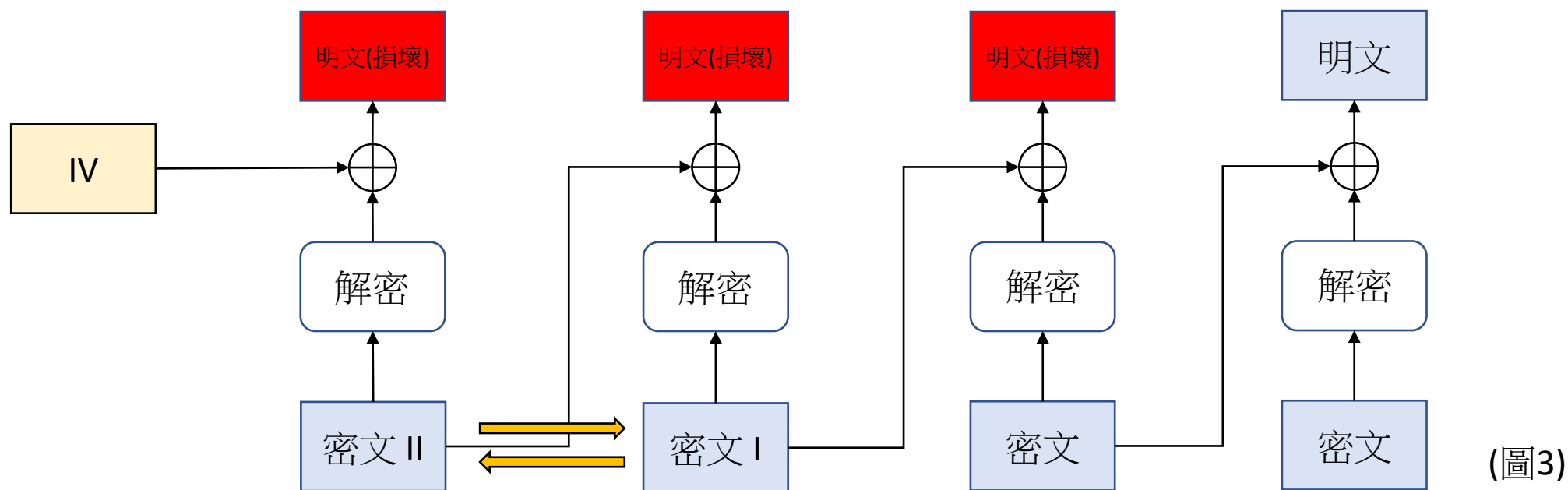
又稱密碼區塊鍊鎖模式，引進了IV (Initialization Vector) 和 XOR 運算使區加密區塊間複雜化並鎖住了密文順序使ECB的缺點不復存在。IV又稱初始向量，為一大小與明文分組大相當的隨機數，一般來說不用特意隱瞞，但要避免多次使用相同金鑰加密同一IV而露出破綻。CBC 模式在一開始先直接將IV用金鑰加密，並將加密結果與第一塊明文做XOR運算，得出第一塊密文，接著再用金鑰加密第一塊密文後與第二塊明文做XOR運算得出第二塊密文，以此類推(如圖2)。

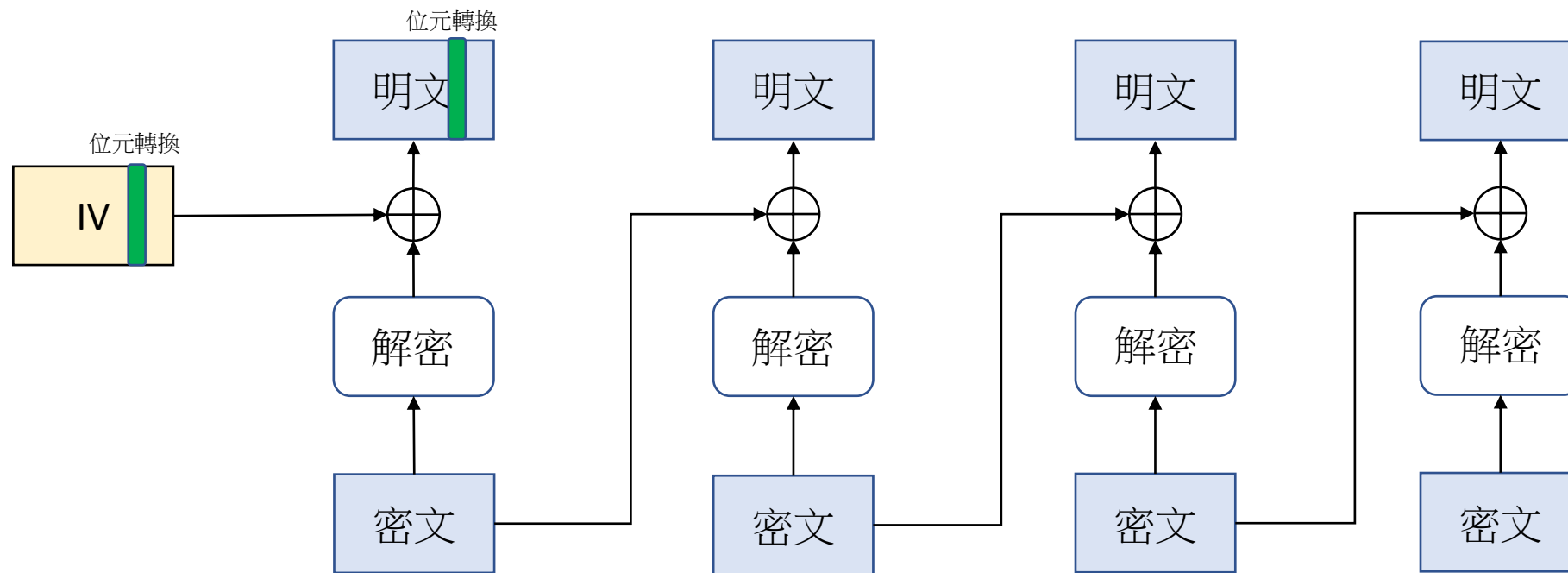


(圖2)

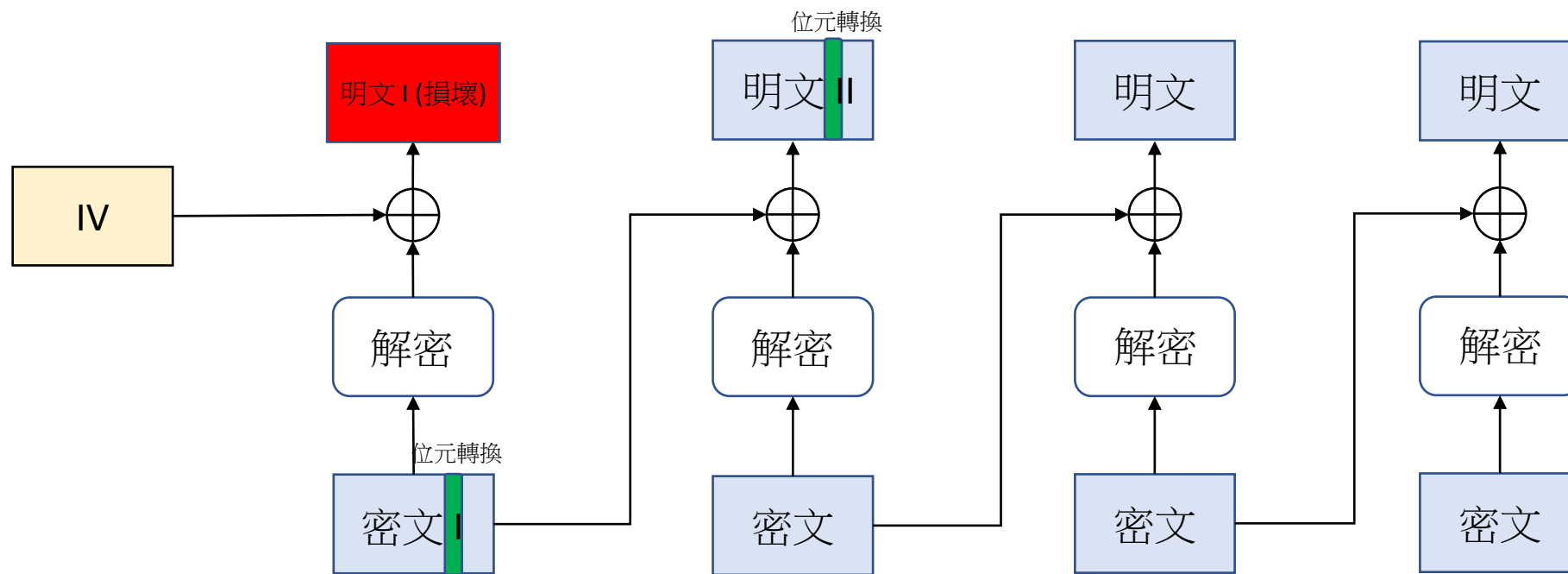
接著來談CBC的優缺點，由於每塊明文加密時都會先與前一塊密文做 XOR 運算，因此就算明文中有一樣的部分也不會像 EBC 一樣有密文重複的狀況，同時也確保了明文加密時的順序，因為如果攻擊者想要將密文調換，在解密時部分明文就會變成亂碼(解密時前一塊的密文與加密時的密文不是同一塊(圖3))。至於缺點的話主要分成幾種

1. 如果某一塊密文發生位元錯誤一次影響的是兩塊明文，而如果因為硬體問題造成某位元遺失，遺失部分後面的部分更是會直接報廢。
2. 第二點是關於XOR的問題，如果將初始向量的某部分做位元反轉，其結果會直接影響到明文的相對位置(因為途中未經過解密，固為原位置不變(圖4-1))，所以如果攻擊者就能以此改變明文(前提是攻擊者知道明文內容)，而如果是在密文做位元轉換攻擊的話效果就會相對遜色，因為改變密文 I 雖然能改變 I+1 的明文，卻同時會破壞掉 I 明文(位元轉換經解密會大幅影響明文資料(圖4-2))。



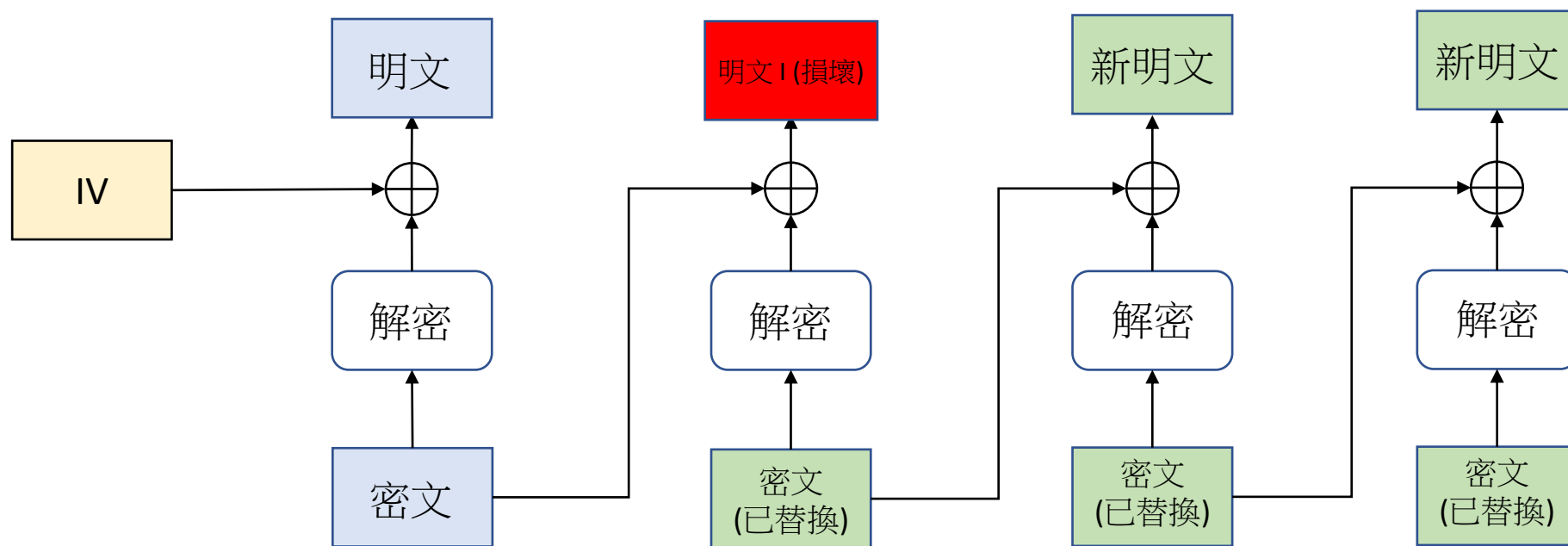


(圖4-1)



(圖4-2)

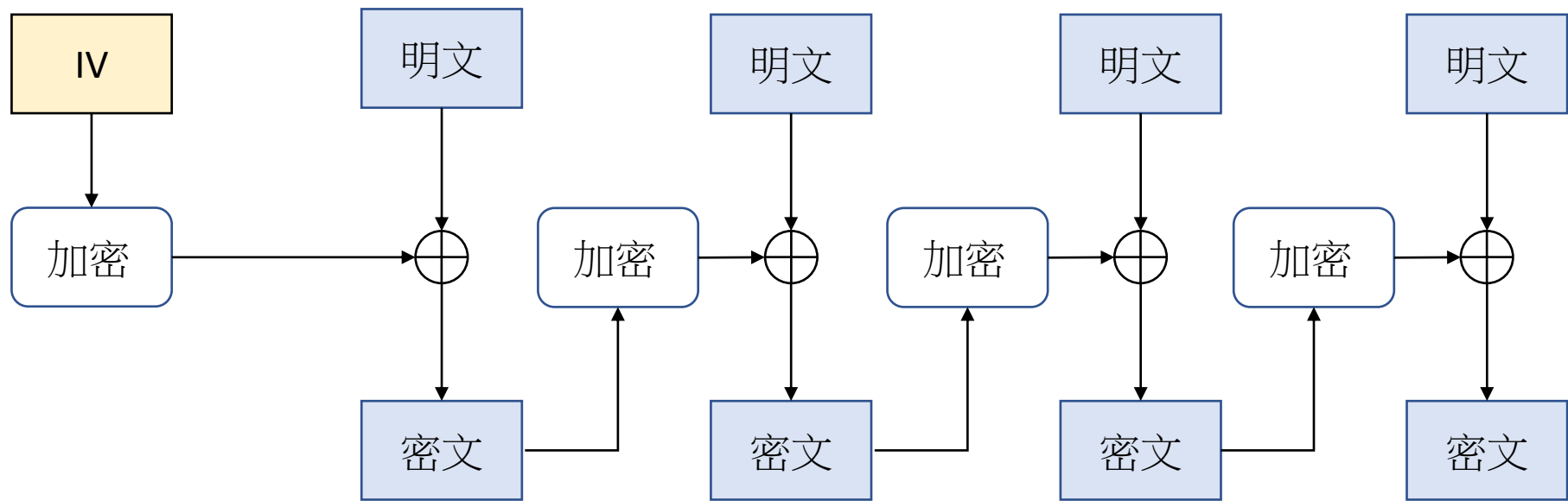
3. 如果多次通訊都用同一把金鑰和初始向量，就能以一塊明文解密錯誤的代價成功混入舊訊息(如圖5)，進而在不破解金鑰的情況下修改明文(畢竟只有一小塊錯誤，很難分辨是人為修改還是通訊錯誤)。
4. 在區塊加密中，當明文長度非加密長度的整數倍時，需要在資料的最後進行填補資料(Padding)，而這項動作也成為了讓攻擊者有機可乘的漏洞。攻擊方式為不斷修改填補資料的值並傳送給收件者(伺服器)，而當伺服器無法正確解密時就會回傳錯誤訊息，利用這種方式能得知一部份的明文資訊，並且不僅限於 CBC 模式，只要有位元填補步驟的模式都有此疑慮(SSL 3.0 過去曾因此遭受嚴重災情)。
5. 由於每次加密時都需要前一塊的密文參與，因此加密時每隔加密塊無法同時進行，但是解密時可以。



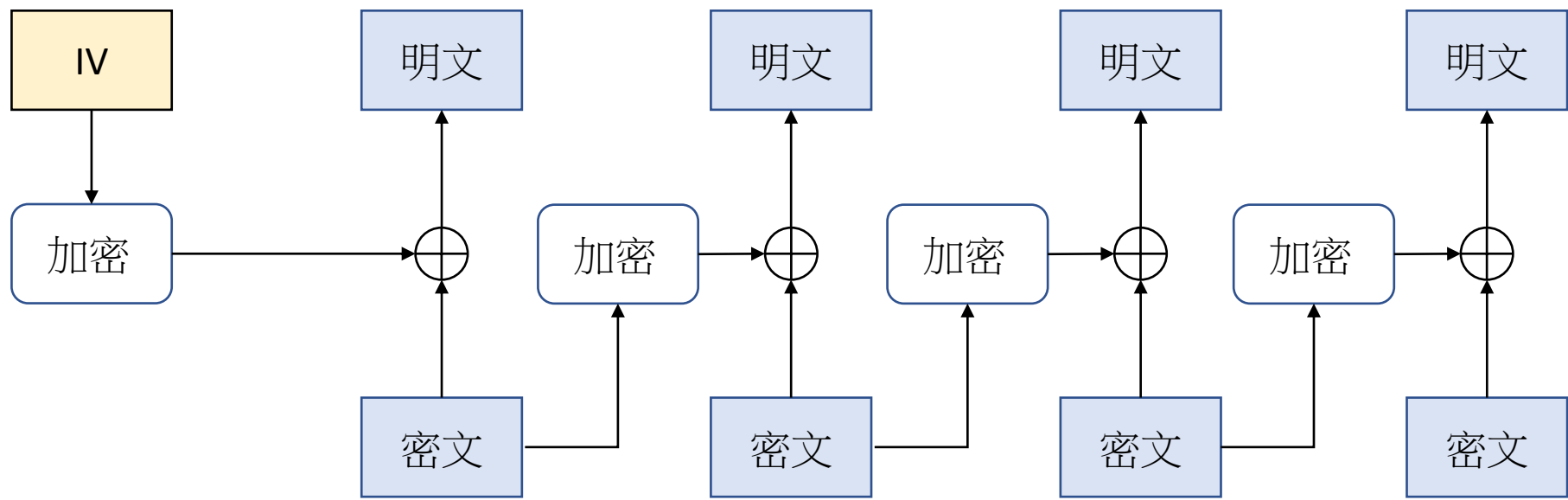
(圖5)

CFB (Cipher-FeedBack mode)

又稱密碼回饋模式，模式運作原理是將前一塊密文當作加密演算法的輸入，而其中的回饋是回歸成輸入的意思。不同於 EBC 與 CBC 利用密碼演算法來加密，CFB 並不是用加密演算法來加密，而是先將前一塊密文輸入加密演算法加密，再將輸出與明文做 XOR 運算(如圖6-1)。既然與 CBC 一樣加密需要前一塊密文的參與，CFB 也需要 IV。由於每次與明文做 XOR 的資料都是前一塊密文決定的，所以 CFB 的加密一樣無法同步進行。還有值得注意的是由於 CFB 模式的明文加密是透過 XOR 運算產生的，所以在解密時同樣需要透過加密演算法的金鑰將 IV 或是前一塊密文加密得到各塊加密的 XOR 金鑰(CFB的加解密皆沒有用到加密演算法的解密功能(如圖6-2))。而這種每次都與不同的資料進行 XOR 來加密雖然與一次性密碼本加密相似，但畢竟 XOR 金鑰是由前一塊密文產生而非隨機產生，所以並非不可能破解。



(圖6-1)



(圖6-2)

優點：

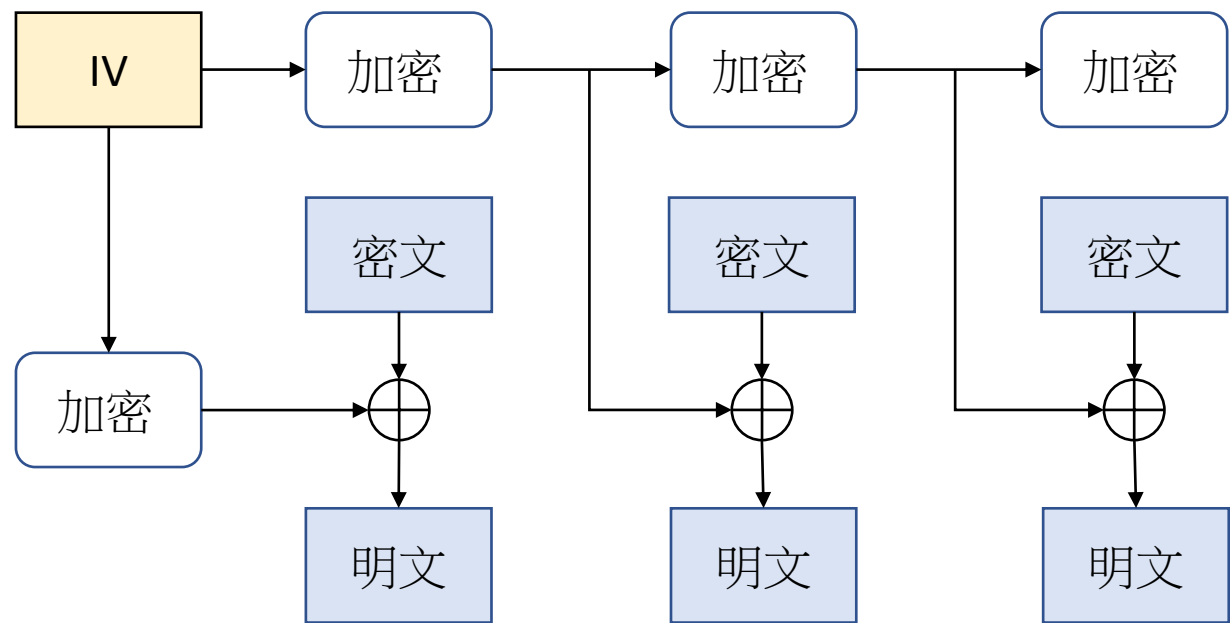
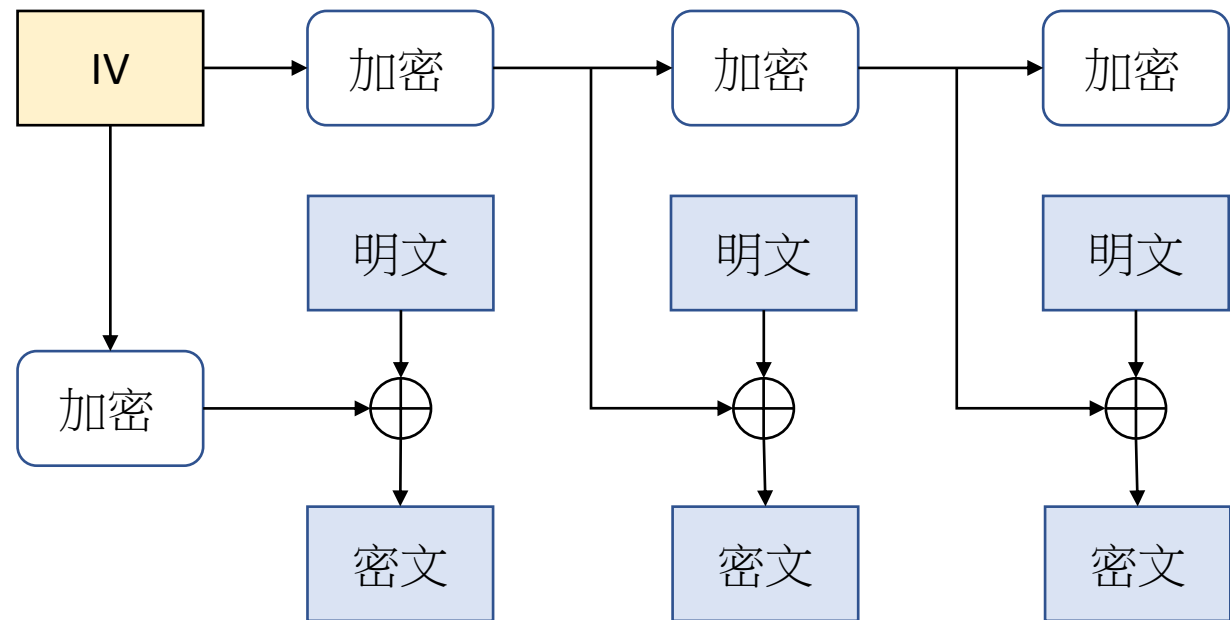
1. 不需要進行位元填補(加解密皆未經過加密演算法處理，因此位元長度不受限)
2. 只需加密器不須解密器，加密兩次即達到解密效果

缺點：

1. CBC的缺點(3)一樣存在並未改進
2. CFB 和 CBC 在加密時每個加密塊無法同步進行

OFB (Output FeedBack mode)

又稱輸出回饋模式，這裡的回歸是指密碼演算的輸出回歸到輸入。與 CFB 相同在於明文加密並非通過加密演算法，而是透過 XOR 運算。不同在於加密演算法輸入的資料(XOR 金鑰的來源)不同，有別於 CBF 依靠前一塊密文來產生 XOR 金鑰，OFB 直接利用前次的 XOR 金鑰來繼續做加密演算法的輸入(下一次 XOR 金鑰的來源)，但是既然 XOR 金鑰還是靠著前次的資料來產生，OFB 依然需要 IV。在我看來，我們可以將 IV 看做生成元，加密演算法當作產生器，而經產生器輸出的結果就是 XOR 金鑰。步驟如下，一開始將 IV 經產生器輸出 XOR 金鑰 1，將明文 1 與 XOR 金鑰 1 做 XOR 運算產生密文 1，接著將 XOR 金鑰 1 再丟入產生器輸出 XOR 金鑰 2 再加密，以此類推(如圖7-1)，我們可以發現其實整個加密過程產生的 XOR 金鑰 i 一開始就被 IV 給決定好了，並且 IV 是影響各 XOR 金鑰的唯一因素(假設使用相同加密演算法)。相對的，解密的架構與加密幾乎一模一樣，只是將明文與密文交換位置而已(如圖7-2)。



優點：

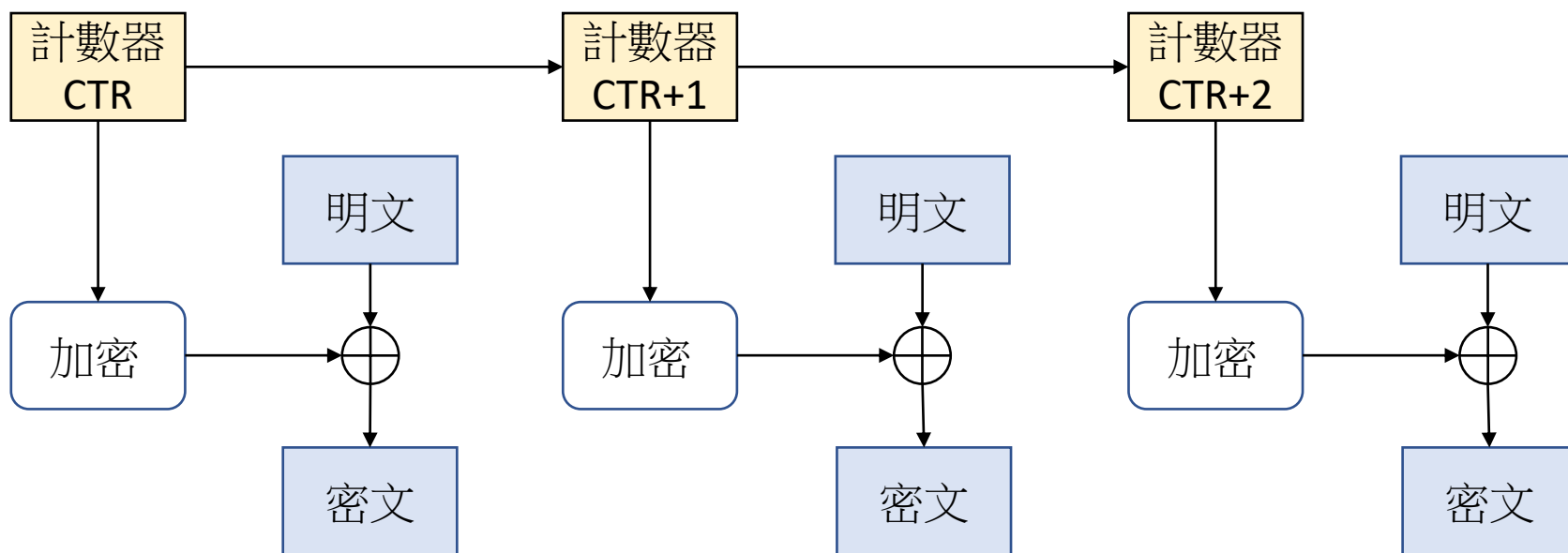
1. 與 CFB 一樣不需填補位元
2. 雖然依然不能進行同步處理(加解密都是)，但可以預先產生 XOR 金鑰，等收到密文即可直接解密 (XOR 金鑰的產生只要有加密演算法的金鑰和 IV 即可產生)
3. 與 CFB 一樣只需加密演算法的加密運算即可完成加解密，除此之外，OFB 自身的加解密結構是一樣的。
4. 出現位元錯誤的密文解密後，只有相對應的明文會受到影響

缺點：

1. 接續 OFB 優點 4，雖然只影響相對應的明文，但影響的位元位置也是相應的，這會使其更容易遭受位元轉換攻擊。
2. 由於 IV 某種程度代表著所有 XOR 金鑰，所以重複使用 IV 將有利於攻擊者抓到破綻。

CTR (Counter mode)

又稱計數器模式，原理是把技術器一個個增加的技術器加密，藉此產生串流金鑰的加密模式。簡單來說就是先利用隨機數(nonce)產生計數器，計數器在一回合作加密及記數的動作，加密計數器後的值在與明文做 XOR 運算產生密文(圖8)。到這裡我們能發現CFB、OFB和CTR都是利用加密器產生 XOR 金鑰來與明文做 XOR 運算完成加密，差別只在於加密器的輸入來源以及 CTR 不需要 IV(被計數器取代)。

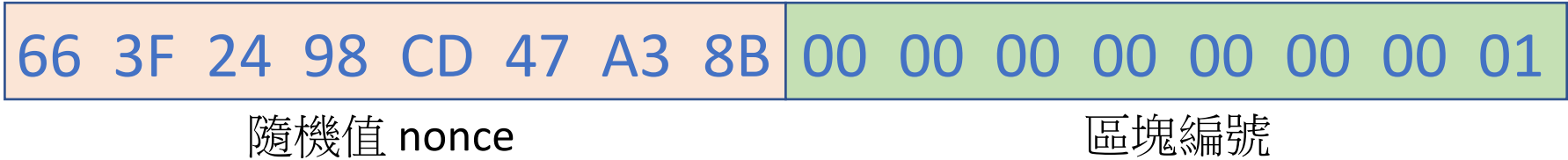


(圖8)

而解密時的結構也完全一樣，只是將密文與明文的位置互換，至於 CTR 的特點是作為加密演算法輸入端的計數器是可以被預測的，也因為此特點讓 CTR 在加解密時能完成並行的運作。

計數器的生成

計數器的預設值是根據每次加密時的隨機值(nonce)來決定，假設區塊長度為 128 位元(16 bytes)，計數器的格式如圖9表示

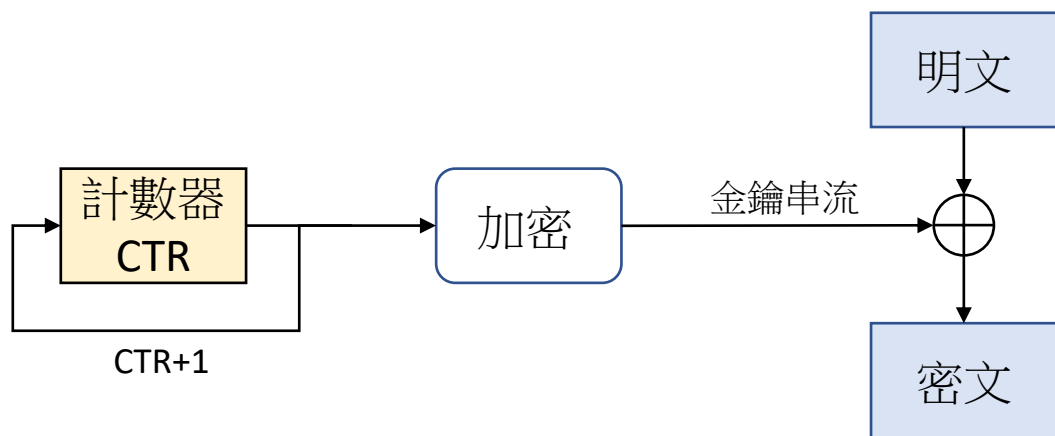


(圖9)

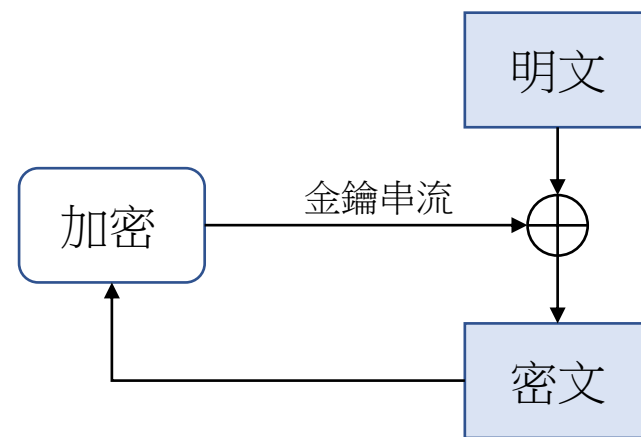
從圖可知，前 8 個 bytes 是表示本次加解密使用的隨機值，而後 8 個 bytes 則表示當下加密的編號，換言之，使用者能事先知道每個加密塊對應的計數器，進而得到相應的 XOR 金鑰，例如，上面範例的區塊編號尾二碼為 01，我們就可以知道下一塊計數器編號的尾二數就會是 02，以此類推 (在同一次加解密中，隨著計數器的計數增加，隨之增加的只有區塊編號的部分，隨機值並未改變)，這也是為何 CTR 能實現加解密的並行。

雖然 CTR 的優缺點與 OFB 大同小異，但有一點是 CTR 優於 OFB 的，我們知道 OFB 的 XOR 金鑰是加密金鑰串流的其中一個區塊所產生的結果，因此如果加密完的 XOR 金鑰不幸與加密前相同，後續的 XOR 金鑰也會跟著完全重複同一個值進而喪失安全性，而 CTR 則不用擔心這一點。

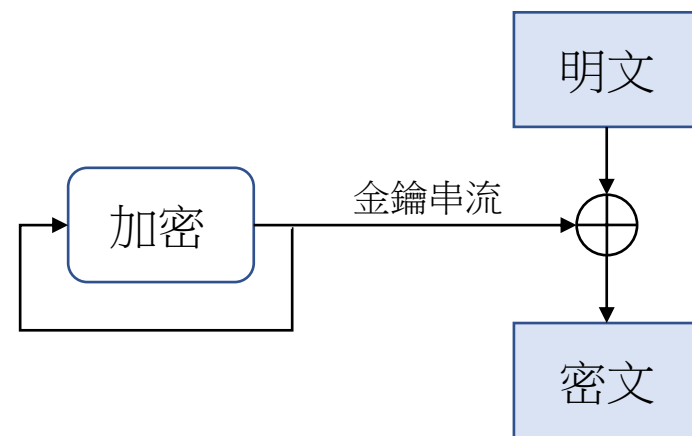
比較 CTR/OFB/CFB 串流金鑰的差異



CTR



CFB



OFB

小結

上述所介紹都是現在主要且常見的模式，因為各個都有自身獨特的優缺點，所以應用的判斷主要是依據使用的環境及優缺點的衡量，下面整理了平時使用時會考慮的方面(圖10)及重要優缺點的整理(圖38)。但如果不去考慮傳送誤差或加解密並行的話，**CBC** 是目前主流的選擇。但是在上述的介紹中我們會發現不管哪個模式都還是無法非常安全的防範攻擊者做一些調換密文或破解的動作，因此，如果要確定我們所收到的密文是否被他人修改過，就必須用到我們後面會提到的訊息驗證碼和身分驗證的保護。

特點	加密模式
區塊加密	EBC CBC
串流加密	CFB OFB CTR
不允許傳遞誤差(資料不能有缺失)	CBC CFB
允許傳遞資料誤差	EBC OFB CTR
可並行	EBC CTR
不可並行	CBC CFB OFB

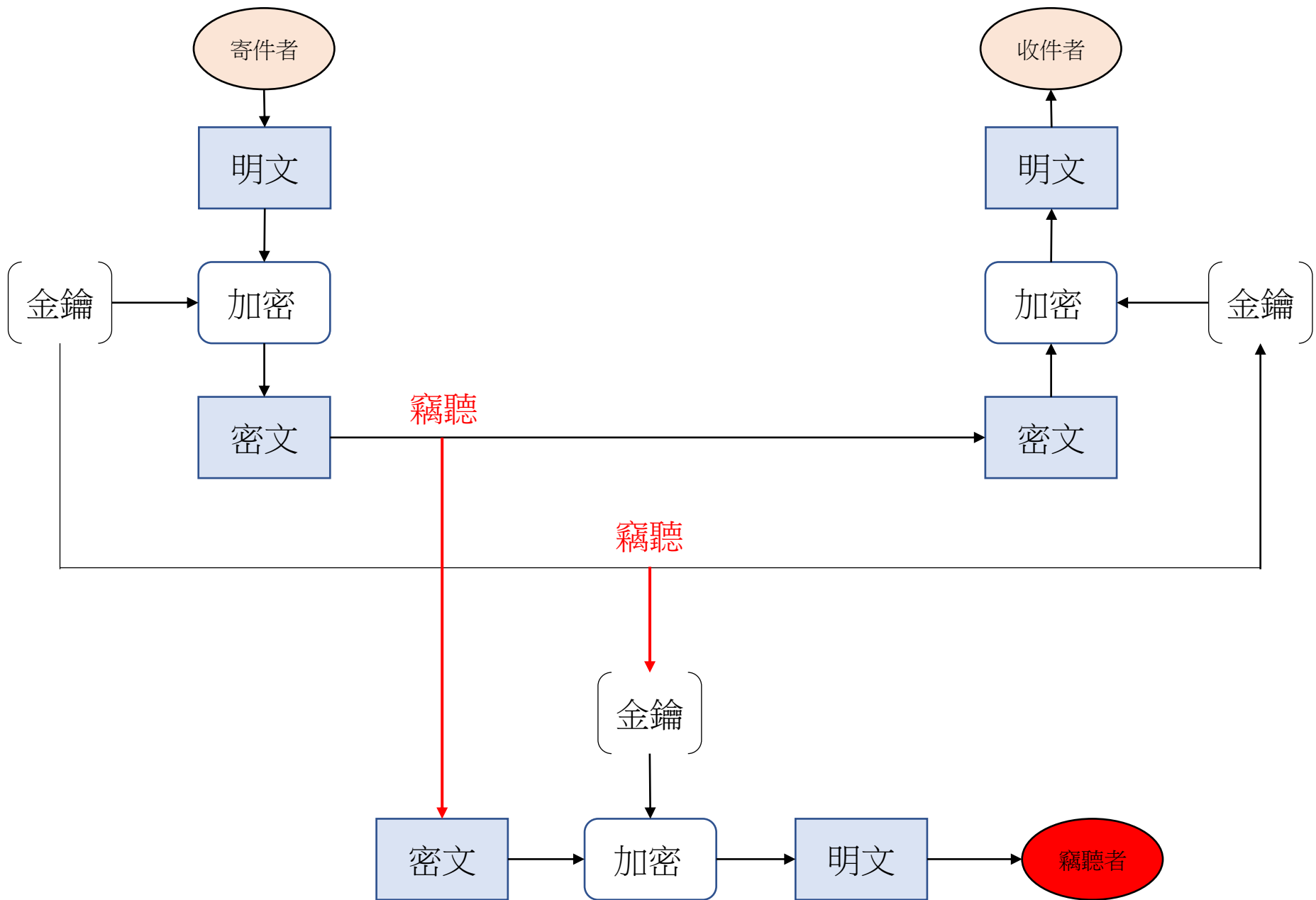
(圖10)

名稱	優點	缺點
EBC 電子密碼書模式	- 簡單 - 快速 - 加解密皆可進行並行處理	- 明文重複部分會反映在密文上 - 可利用刪除或調動密文來修改明文 - 可能遭受重送攻擊
CBC 密碼區塊鏈結模式	- 明文重複部分不會反映在密文上 - 僅解密可行並行處理 - 可解密任意密文區塊	- 加密無法進行並行處理 - 出現錯誤的密文塊在解密時會連帶影響下個區塊的明文 - 可能遭受重送攻擊
CFB 密碼回饋模式	- 不須進行填補處理(padding) - 解密時可並行處理 - 可解密任一密文塊	- 加密時無法並行處理 - 出現錯誤的密文塊在解密時會連帶影響下個區塊的明文 - 可能遭受重送攻擊
OFB 輸出回饋模式	- 不須進行填補處理(padding) - 可事前準備加解密需要的 XOR 金鑰 - 加解密結構相同 - 出現位元錯誤的密文解密後只會影響相應的明文	- 加解密皆無法進行並行處理 - 可能遭受位元反轉攻擊
CTR 計數器模式	- 不須進行填補處理(padding) - 加解密結構相同 - 出現位元錯誤的密文解密後只會影響相應的明文 - 可事前準備加解密需要的 XOR 金鑰 - 加解密皆可並行處理	可能遭受位元反轉攻擊

公開金鑰

在對稱式金鑰系統中，安全的傳送金鑰給收件者是一件困難事，如果毫無保護措施的將金鑰發出去很可能會因為被攻擊者監聽而造成加密的訊息被一覽無遺(圖11)。因此，我們先來探究幾種處理上述金鑰配送問題的方式

- 利用事先共用金鑰
- 借助金鑰配送中心
- 利用 Diffie-Hellman 金鑰交換來解決問題
- 利用公開金鑰密碼



(圖11)

事先共用金鑰

最簡單且土炮的方式，就是以安全的方式事先將金鑰交給對方並且避免使用網路或郵寄等中途會被攔截的方式，例如直接面對面的交給對方，但如果雙方有著物理上的距離，此方法也會窒礙難行。就算使用者都是同一間公司的人，此方式也不會是上策，假設一家公司有**1000**名員工，如果每位員工彼此間進行加密通訊(假設每兩位員工都有一把獨立的通訊金鑰，畢竟誰也不能確保公司裡有間諜)，則每位員工都都有**999**把鑰匙，以此類推，整間公司需要有 $1000 \times 999 \div 2 = 499500$ 把鑰匙。因此，事先共用金鑰雖然理論有效，但使用上卻有諸多限制。

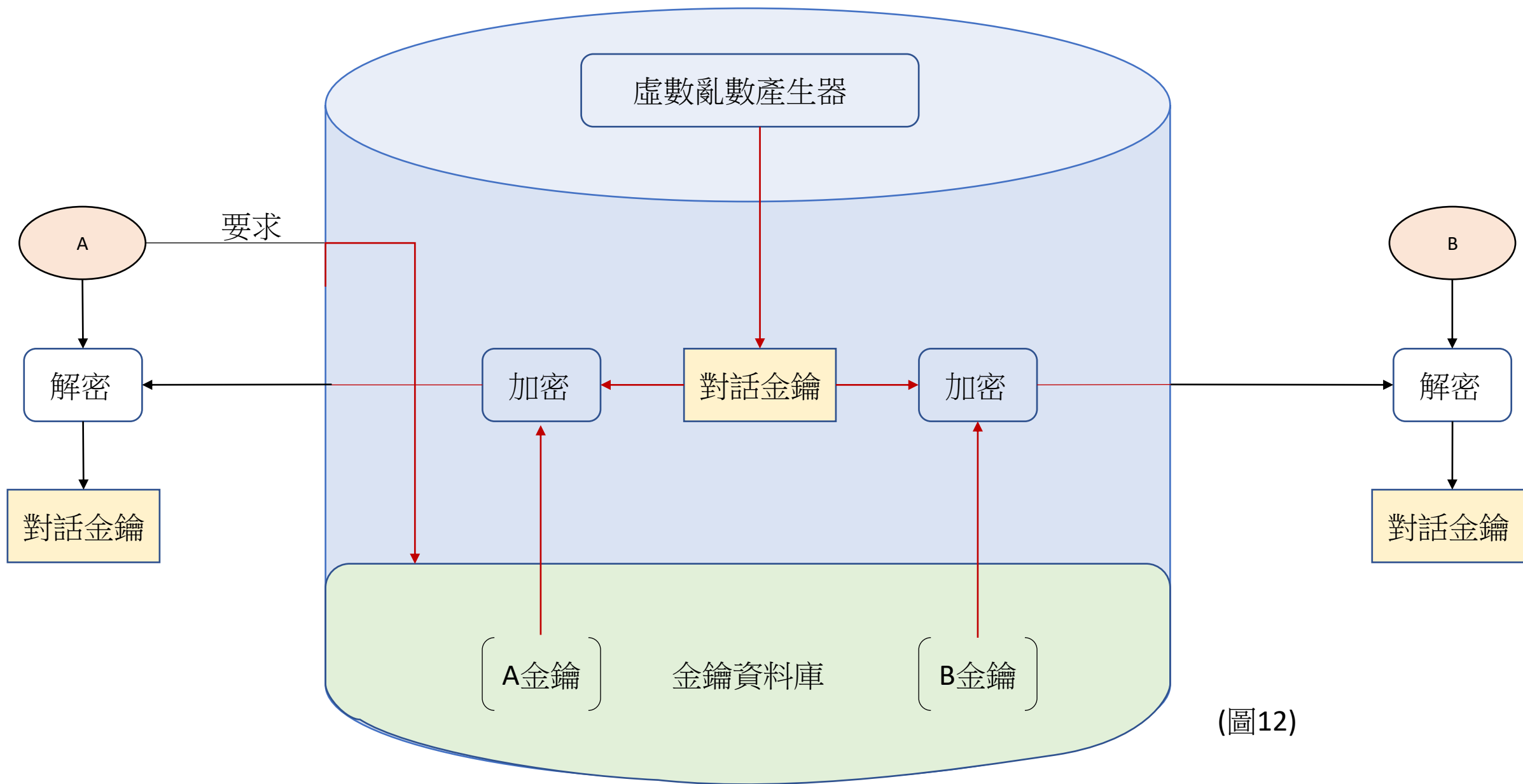
借助金鑰配送中心

這種方式是在每次進行秘密通訊時，由金鑰配送中心產生當次通訊用金鑰，而使用者只要事先與配送中心共用金鑰即可。所以這樣能有效減少金鑰數量爆炸問題，因為配送中心只要儲存相當公司總人數的金鑰即可讓所有人互相通訊，而如果有新進員工，也只要在資料庫中心新增資料即可。

假設現在 A 想和 B 通過配送中心通訊(圖12)，流程如下

- (1) A 向配送中心提出與 B 通訊的請求
- (2) 金鑰配送中心使用虛數亂數產生器產生對話金鑰(session key)，為 A 和 B 此次通訊用的金鑰
- (3) 金鑰配送中心重資料庫中取出 A 和 B 的金鑰並用其加密對話金鑰，再將密文傳給 A 和 B
- (4) A 和 B 在收到密文後分別用自己的金鑰解密，得到對話金鑰
- (5) A 和 B 使用對話金鑰開始與對方通訊

雖然此方式比事先共用金鑰好上不少，但還是有使用上的限制。首先，如果使用人數未分流或是其他原因導致配送中心癱瘓或是故障，那資料庫中的每個用戶都會受到影響，再者，如果配送中心並未做好安全防護導致被入侵，那所謂的安全通訊將直接被瓦解，因為攻擊者已取得資料庫中所有人的金鑰



(圖12)

Diffie-Hellman

Diffie-Hellman 金鑰交換簡單來說是一種能讓想祕密通訊的兩人，彼此交換資料，而這份資料就算被監聽也沒關係，因為這份資料是只有通訊的兩人才能產生出相同的金鑰，聽起來有點抽象，後面會再做說明

公開金鑰密碼

在對稱式加密系統中，加解密用的是同一把金鑰，而本次要介紹的公開金鑰系統就是特殊在加解密用的金鑰並非同一把，而在做通訊時只要將公開金鑰傳送給收件者即可，與對稱式加密一樣，不同的是當收件者回傳密文時，監聽者僅握有公開金鑰是無法解密訊息的，只有握有解密金鑰的發送人才能解密以自己的公開金鑰加密的密文，進而解決金鑰配送問題。聽起來有點混亂，所以下面我將詳細介紹

公開金鑰密碼 (public-key cryptography)，又稱非對稱密碼，特點為將加密金鑰與解密金鑰分開，通常會在通訊過程中流通的金鑰我們稱為公開金鑰(又稱“公鑰”)，是用來加密訊息的，而使用者自己保存的就是解密金鑰(又稱“私鑰”)，用來解密訊息的。整理成下列兩點

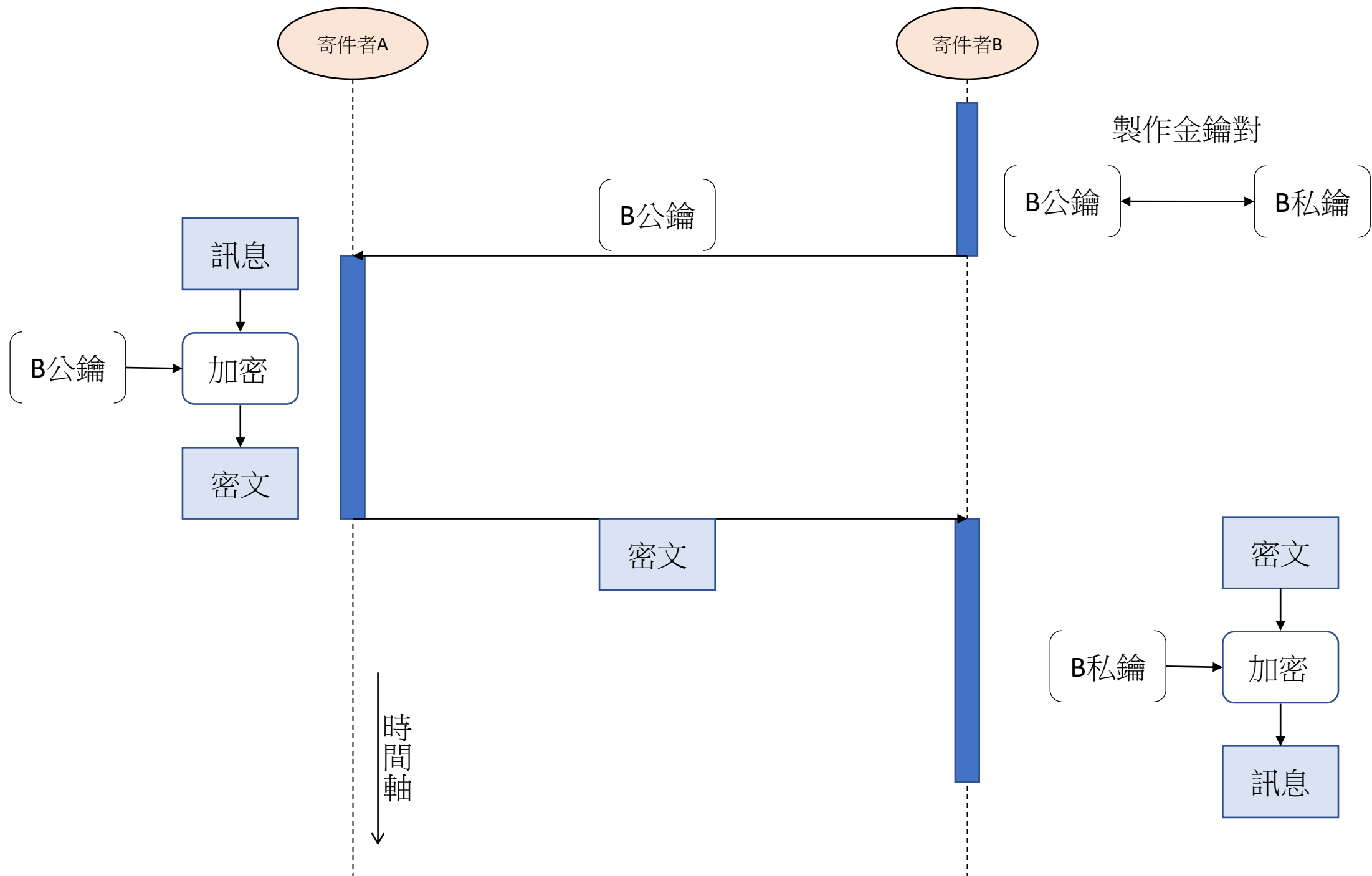
- 加密訊息者只需要“加密金鑰”，解密訊息者只需“解密金鑰”
- “解密金鑰”不能被竊聽者知道，而“公開金鑰”可以

公開金鑰與私密金鑰是有相應配對的，一對完整的公私鑰稱為“金鑰對(key pair)”，成為金鑰對的公私鑰彼此有著密切的數學關係，且無法只單獨產生公鑰或私鑰。

公開密碼的通訊過程

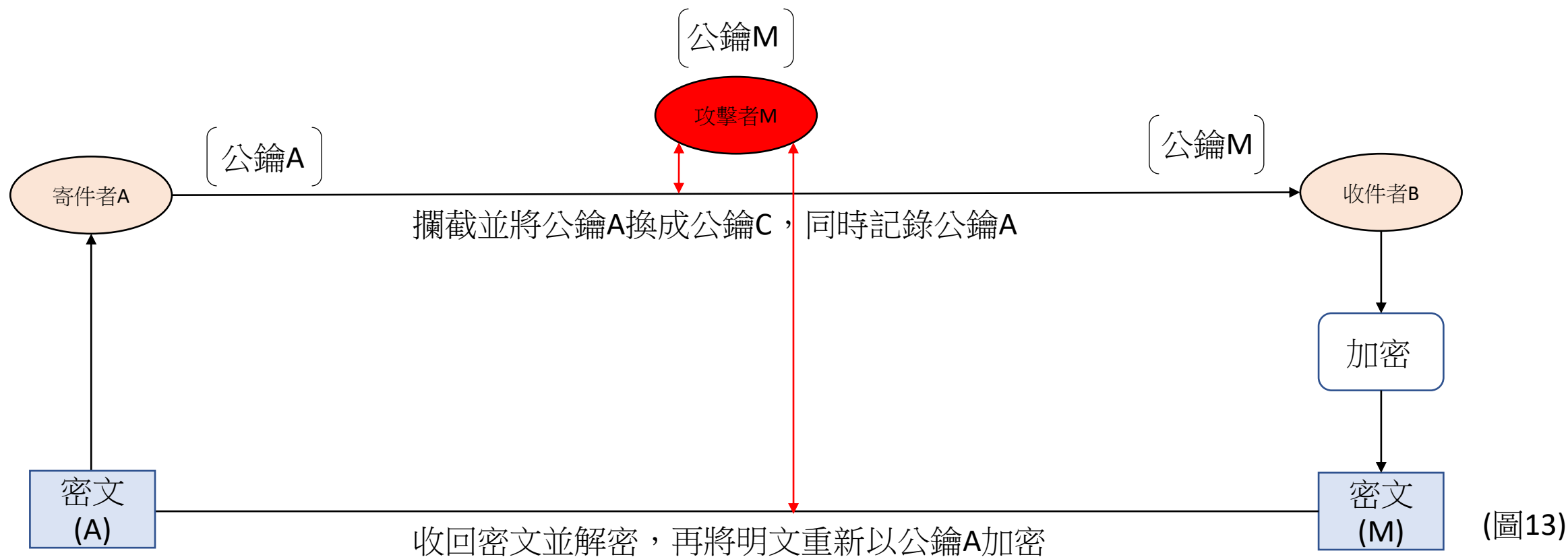
一樣假設 A 要傳送訊息給 B

- (1) B 先製作金鑰對，並保存好私密金鑰
- (2) B 將公開金鑰傳送給 A，而 A 再利用此公鑰將要傳送的訊息加密後回傳給 B
- (3) B 利用自己的私鑰解密 A 傳來的密文



公開密碼無法解決之問題

儘管公開密碼能解決金鑰的配送問題，但並不是一切問題都迎刃而解，因為如果僅僅是使用公開密碼系統通訊，就無法確定自己收到的公鑰是否正確，進而可能遭受中間人攻擊(圖13)，因此需要配合其他方式來解決公開密碼系統之授權問題。



RSA

RSA是一種常見公開加密演算法，主要利用數學中的大型質數分解和離散對數難題來維持其安全性，相較於之前提到的對稱式加密，**RSA**感覺就像是躲在名為數論難題的天然堡壘裡面，因此其加解密相較於位元的打散更為直觀，但同時也會增加電腦的負擔。

RSA的加密

在 **RSA** 中，明文與密文都是由數字組成而並非 1 與 0 的位元，並在加密時遵守以下算式：

$$\text{密文} = \text{明文}^E \bmod N$$

跟前面的對稱式加密或雜湊比起來簡直是天壤之別，如同前面說的，**RSA**的安全性靠的是數學的難題，因此促成這條加密算式的重點就是 **E** 和 **N**，而且 **E**、**N** 還有等等解密用到的 **D** 都是經由特別計算所得的特別數字而非隨便選的。接著來看解密算式：

$$\text{明文} = \text{密文}^E \bmod N$$

我們可以發現其實架構與加密的算式是差不多的，只是指數從 E 換成了 D 。RSA 在生產公私鑰時總共會產出 3 個數字，分別就是E(Encryption) 、 D(Decryption) 和 N(Number)，而前面所說通訊時要寄出的公鑰在 RSA 中就是指公鑰對 {E , N} ，而自己留存的就是私鑰對{D , N}，並且二者之間是環環相扣的，如果公私鑰配不起來就無法完成加解密

產生金鑰對

接著我們來看如何生成一組包含 E 、 D 和 N 的完整金鑰對，過程如下

(1) 計算N

首先我們要準備 2 個大質數並分別先用 p 和 q 來表示，p 和 q 如果太小很容易就被破解，但如果太大運算時又會太占資源，因此通常 p 和 q 會取 1024 位元(約莫 309 位數)，而要產生如此巨大的質數需要用虛擬亂數產生器產生擁有 1024 位元的數字之後再逐一篩選出質數，準備完 p 和 q 之後，將這兩數相乘就能得到金鑰對的 N 了

$$N = p \times q$$

至於為什麼 N 是由 p 和 q 相乘關係到許多數學知識，我曾經去網路上找過相關資料，但是有看沒有懂，所以這裡我就先跳過了。

(2) 計算 L (只有在產生金鑰對的時候才會出現)

接著要計算出 L 的值，L 只有在製作金鑰對的時候才會出現。而 L 就是 $p - 1$ 與 $q - 1$ 的最小公倍數

$$L = \text{lcm}(p - 1, q - 1)$$

(3) 計算 E

之後我們就可以開始算出加密金鑰對的 E 了，E 必須是大於 1 但小於 L 的數字，並且同時 E 和 L 要互質，而通常產生的方式也是利用亂數產生器先找出範圍內的質之後再去驗證是否互質。簡單講，E 和 L 之所以要互質是因為要確保解密用的金鑰 D 存在。

$$1 < E < L \ \&\& \ \text{gcd}(E, L) = 1$$

(4) 計算 D

最後我們能藉由 E 和 L 求出 D，關係如下：

$$1 < D < L \ \&\& \ E \times D \bmod L = 1$$

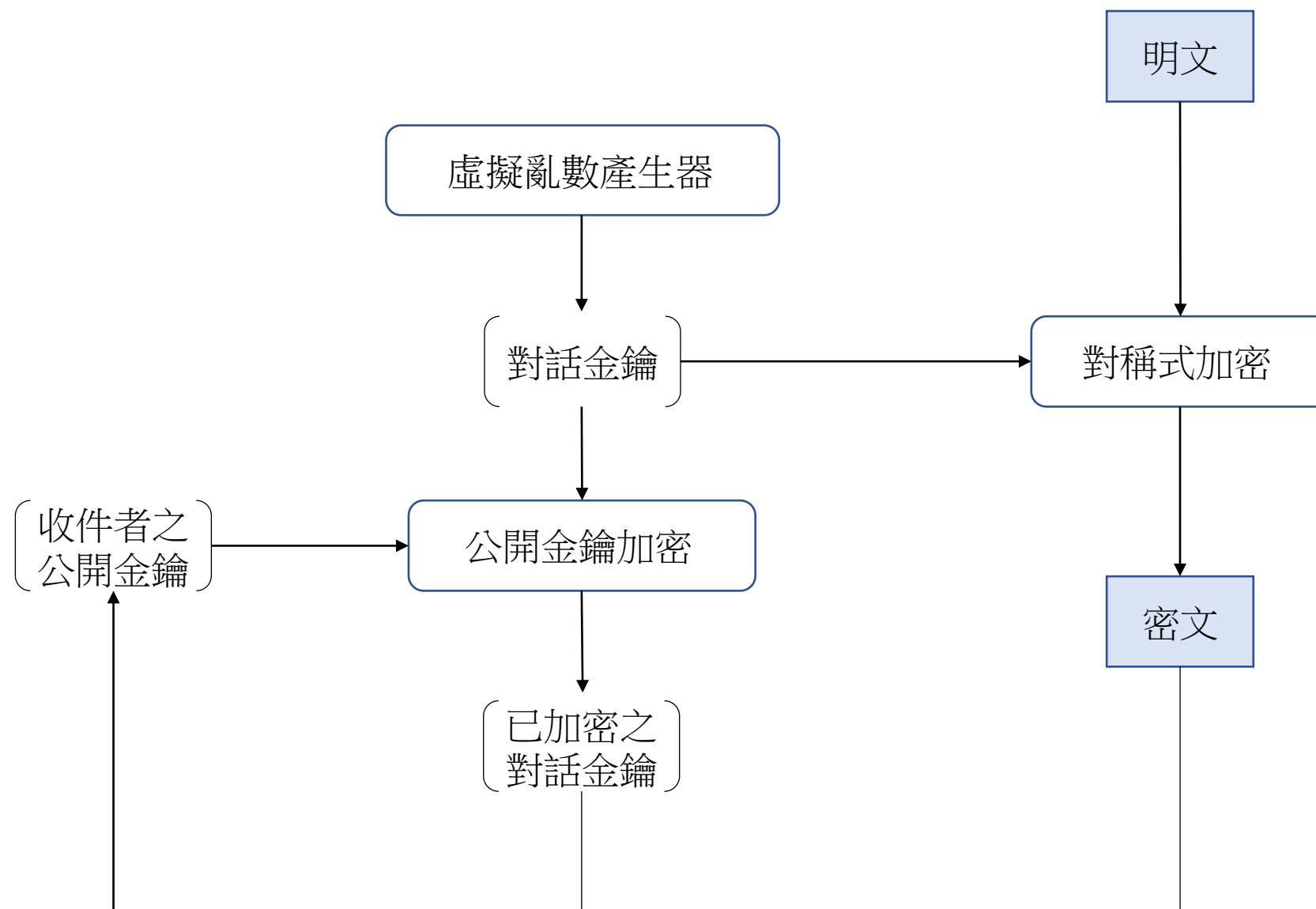
找到滿足條件的 D 之後，就能將 E 和 N 加密的密文利用 D 和 N 來解密。 $E \times D \bmod L = 1$ 就是可以將密文恢復成明文的保證。

混和型密碼系統

混和型密碼系統(hybrid cryptosystem) 各取了對稱式加密及公開密碼的優點，先用處理速度較快的對稱式密碼加密訊息，再利用公開金鑰系統加密對稱式加密的金鑰，保護解密訊息金鑰之機密性，而且由於一般對稱式金鑰的長度都會比明文短，因此不會造成公開金鑰加密時的太大負擔。大致步驟如下

- (1) 利用亂數產生器產生對稱式加密的對話金鑰
- (2) 利用對話金鑰先加密明文
- (3) 再利收件者的公鑰加密對話金鑰
- (4) 將密文及加密過後的對話金鑰一併傳給收件者(如圖13)

其中的對話金鑰是指每次通訊時，亂數產生器所產生用於加密明文的金鑰，由於每次通訊都會有不同的金鑰，因此叫做對話金鑰(有點類似於網頁session的概念)。



(圖13)

了解了加密步驟後，就會知道解密只是到著走一遍而已，唯一需要注意的是收件者需要事先跟寄件者得知如何分割對話金鑰和密文(收件者收到時可能兩者會連在一起)，剩下的就依序解密即可，步驟如下

- (1) 先將密文與加密後的金鑰分割
- (2) 利用私鑰解密得出對話金鑰
- (3) 利用對話金鑰解密密文得出明文

最後討論一下安全性，我們知道混和型密碼是由虛擬亂數產生器、對稱式加密和公開金鑰加密所組成，而後兩個的安全性前面都提過了因此我們簡單討論虛擬亂數產生器，密碼學應用所使用的虛擬亂數產生器有別於網路上抽獎用的亂數產生器，鑰應用於加密的亂數產生器必須更嚴謹的設計，如果設計有疏失而導致攻擊者能預知到下一次的對話金鑰，儘管只有部分位元都有可能對話金鑰空間變窄而被暴力破解，因此亂數產生器雖然不是負責加解密的演算法，卻有可能成為安全的漏洞。

訊息驗證碼

前面我們有提到不管是對稱式加密或是公開金鑰在通訊時，收件人都無法確認收到的訊息是否真的是寄件者所發或者是遭到竄改，一條安全且未受竄改的訊息應該要保有兩個性質，“完整性”和“認證”。訊息具有完整性就是指訊息的內容並未受到任何修改，而這可以透過前面所介紹的雜湊值來幫助檢查。但僅有保持完整性是不夠的，因為無法確認寄件者的真實身分，因此，認證也成為了驗證訊息安全性的重要一環，訊息的認證指的是訊息由真正的寄件人所發出的特性，且能夠防範他人偽造寄件者。

什麼是訊息認證碼？

訊息認證碼(message authentication code) 指的是能確認訊息完整性且進行驗證的技術，其原理簡單講就是利用雜湊函數來確認訊息完整性，再利用寄件及收件者雙方共同擁有的共用金鑰來認證訊息，也可以說是需要金鑰的單向雜湊函數，而最後輸出的訊息認證碼簡稱 **MAC**。

訊息認證碼的使用步驟

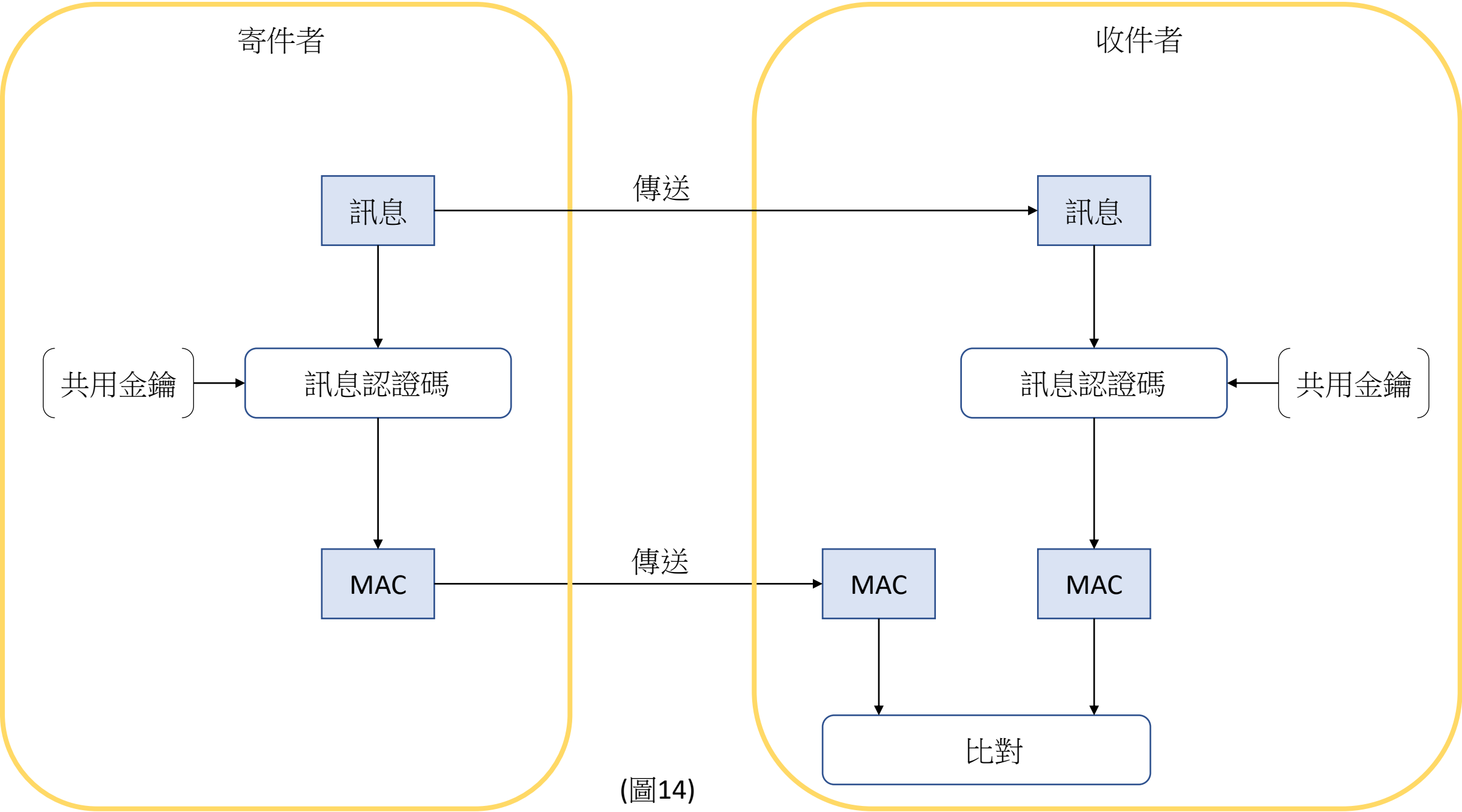
- (1) 寄件者或收件者先產生一把共用金鑰，再利用安全的傳送方式將金鑰寄給對方
- (2) 寄件者根據明文產生 MAC 值(使用共用金鑰)
- (3) 寄件者將明文及 MAC 值一併傳給收件者
- (4) 收件者依照收到的明文在計算出 MAC 值並與收到的 MAC 值做比對
- (5) 如果兩者相等級認證成功，反之則失敗 (如圖14)

金鑰配送問題

根據上述我們可以知道要完成 MAC 驗證需要與對方共用一把金鑰，所以如何在通訊中保護共用金鑰是維持 MAC 功能的重要環節，因為如果共用金鑰洩漏出去，MAC 值基本等於沒用了。至於如何保護共用金鑰在通訊中的安全可以回去參考公開金鑰系統的方式，再依照不同的情況選擇。

SSL/TLS

SSL/TLS 是我們平時線上購物時所使用的通訊協定，它也一樣使用了訊息認證碼來保障通訊內容的認證及完整性。



(圖14)

建立訊息認證碼的方法

再認識完`訊息認證碼的使用方式後，我們接著鑰來知道剛剛一值提到的 **MAC** 值到底是怎麼計算出來的，但其實訊息認證碼有各種建立方法

(1) 使用單向雜湊函數

顧名思義，使用 **SHA-2** 等雜湊函數來建立訊息雜湊碼，其中有一種方式叫做 **HMAC**，也是等等會較詳細介紹的

(2) 區塊加密法建立

這種方式直接把區塊加密法的金鑰拿來當共用金鑰，並再用 **CBC** 模式將整個訊息加密，最後只取最終區塊當作 **MAC** 值，其餘的捨去。由於 **CBC** 模式的最終區塊會受到整個訊息及金鑰的影響，因此可以做為訊息認證碼

(3) 其他方式

其他方式像公開金鑰系統或串流加密都可以用來產出訊息驗證碼

HMAC

HMAC 是前述的其中一種訊息認證碼產生方式，主要是利用單向雜湊函數來運作，所以其中的 H 就是指 Hash 的意思。

HMAC 的執行程序

HMAC 依照下列方式來產生 MAC 值(如圖15)

(1) 填補金鑰

首先要處理的是金鑰的部分，如果金鑰長度不足雜湊函數的長度就要在後面補 0，使金鑰長度與雜湊值長度一致，反之，如果金鑰長度過長，我們就必須取金鑰的雜湊值做 MAC 運算，所以重點就是要將金鑰處理成雜湊值的長度。

(2) 與 ipad 做 XOR 運算

處理完的金鑰我們稱作 HMAC 金鑰，要與稱作 ipad 的位元串進行 XOR 運算，ipad 是指重複 00110110 的位元串，直到長度與 HMAC 一致。(ipad 的 I 是 inner 的意思)

XOR 運算後得到的值一樣會與雜湊值的長度一樣，並將此位元串稱作 **ipadkey**

(3) 與訊息結合並雜湊

接著我們要將剛剛得到的 **ipadkey** 加在訊息的開頭，再輸入單向雜湊函數做運算，計算出雜湊值

(4) HMAC 金鑰與 opad 進行 XOR 運算

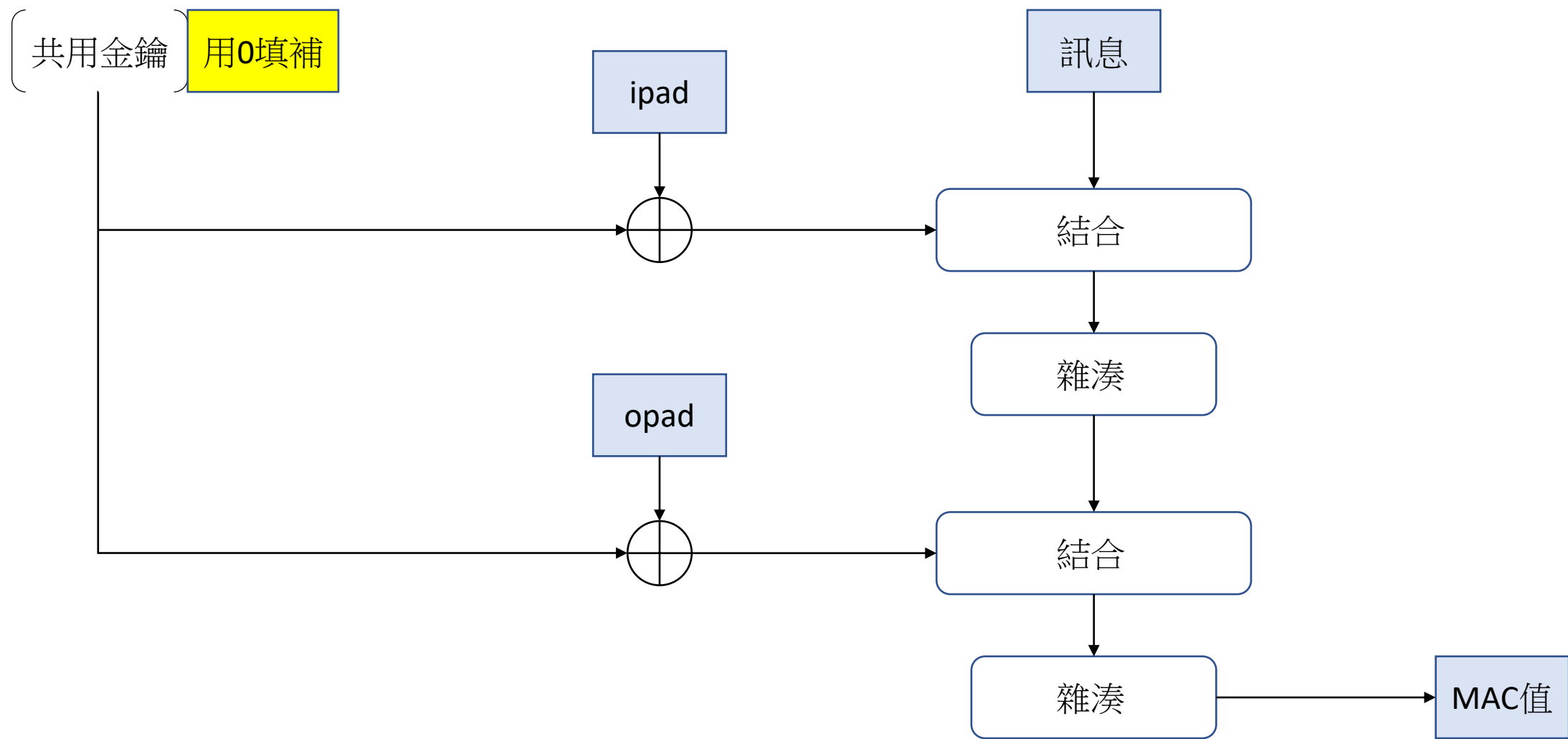
再將前面的 HMAC 金鑰與名為 **opad** 的位元串做 XOR 運算，**opad** 是指重複 01011100 (16 進位為 5C) 的位元串，運算完的位元串稱作 **opadkey**

(5) 結合與雜湊

將 **opadkey** 與 (3) 的雜湊值做結合

(6) 得出 MAC 值

最後將 (5) 的結果輸入單向雜湊函數，得出的雜湊值即為 **MAC** 值



(圖15)

對訊息認證碼的攻擊

重送攻擊

其實就是監聽寄件者所傳出的訊息和MAC值，並原封不動的一值重複傳送給收件者，而如果剛好傳送的内容是類似匯款通知的資訊，那重送攻擊就會奏效，因為攻擊者只是重複傳送寄件者所傳送的有效資訊，所以收件者就算比對MAC值也不會發現異常(先排除收件者注意到資訊被重複傳送)，因此我們需要在計算MAC值時做一些能區別次數的動作

- 加入序號

雙方約定在每次通訊的訊息中都加入隨次數改變的序號，讓每次傳送訊息時都會有不同的MAC值(包括重複的資訊)，但缺點是雙方都需要確保所加入的序號一致

- 加入時戳

與加入序號差不多概念，只是將序號換成時間資訊，但只能每一段時間所傳出的訊息不會被重送，因為通訊的時間需求讓時戳需要一點彈性空間來讓收件方也能使用到一樣的時戳，所以此種方式可能會被攻擊者做短時的重送攻擊。

- 加入隨機數

這個方法是在傳送訊息前先傳送一次性隨機數給對方，而收件者再將隨機數加到等等收到的訊息中計算 **MAC** 值，但是這種方法會稍微增加通訊量

暴力攻擊或生日攻擊

和對單向雜湊函數的攻擊一樣，我們可以透過暴力攻擊或者是生日攻擊法來攻擊，但由於訊息驗證碼的計算中有無法被推知的金鑰，因此攻擊效果會大大的削弱。

MAC 無法解決的問題

雖然 MAC 解決了大部分的假冒以及訊息完整性的問題，但還是有一些其他方面的身分驗證問題存在

- 對第三者的證明

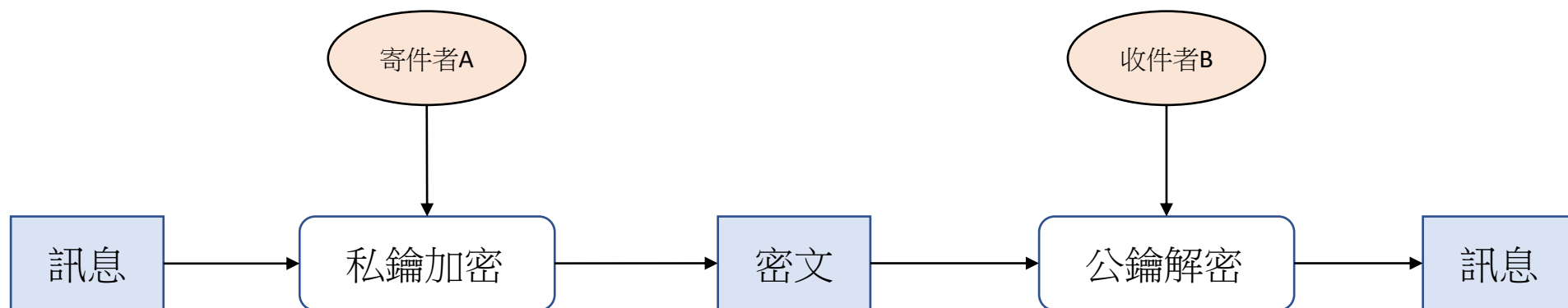
訊息驗證碼解決的身分驗證問題其實僅限於寄收件雙方而已，一旦到了要向非傳輸者驗證時就會失效，因為第三方並不知道兩人通訊用的對話金鑰，所以無法利用 MAC 值確認訊息來源。但如果將通訊範圍擴大，拿到共同金鑰的人數大於兩個時，我們就會發現連寄收件人都會開始無法確認訊息來源，因為知道共通金鑰的寄件者不在只有一個了，進而對內也失去驗證效果

- 防止否認

接著上述，由於持有共同金鑰的人數必會大於兩個，因此無法令訊息具有防止否認性。假如收件者 B 向驗證者 V 聲稱某訊息是由寄件者 A 所寫，但寄件者仍可以聲稱訊息是 B 自己寫的(因為 A 和 B 都有金鑰來寫出這份訊息)，而這種情況就稱為“否認”。

數位簽章

延續前面介紹 MAC 時最後提到的問題，無法防止否認及對第三方證明，所以我們接著介紹能處理這些身分驗證的機制，數位簽章。MAC 無法防止否認的原因在於寄件者和收件者都使用同一把金鑰，所以使用金鑰的人會大於兩人，進而無法確認身分。但如果我們結合公開金鑰系統的特性，使寄件者能用自己專屬的私鑰來產生類似簽名的效果，再由收件者經由其公開的公鑰來解密得到寄件者身分的保證。這樣就能有效解決前述的兩種驗證問題了，因為寄件者所加密的私鑰是別人無法得知及使用的 (除非被偷了)，因此我們就能得知訊息確實由正確寄件者所寄，且寄件者無法否認。



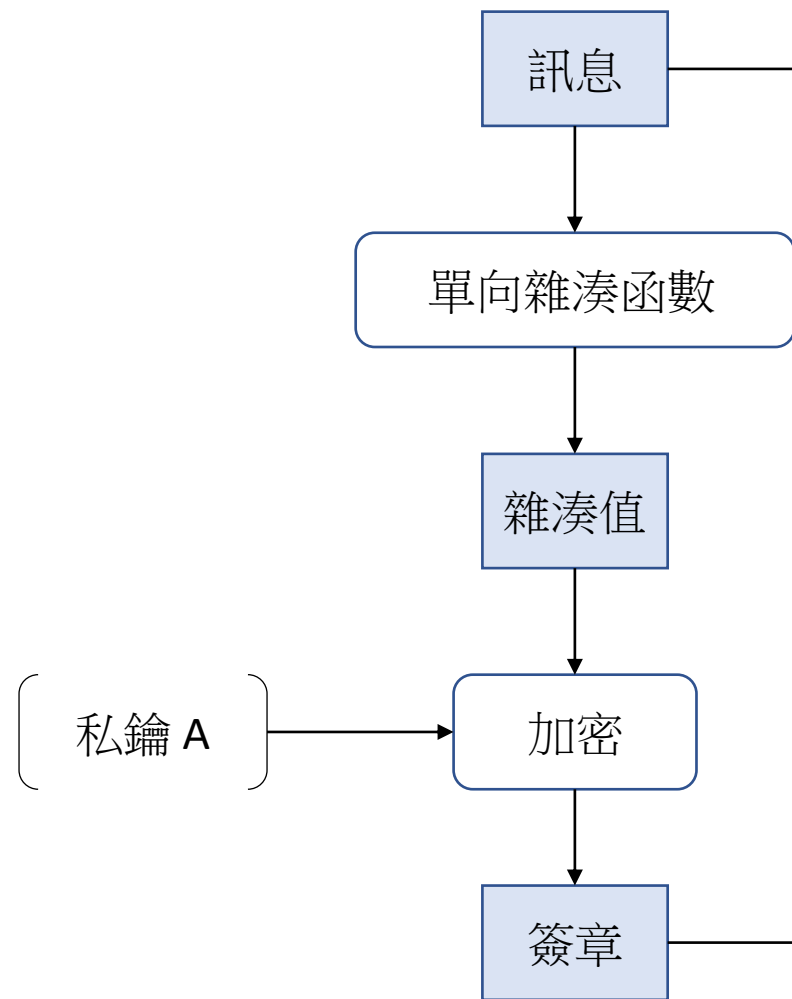
加上數位簽章方法

通常我們為了節省時間及資源並不會將整段訊息加密做成數位簽章，而是直接加密訊息所產生的雜湊值產生數位簽章，步驟如下

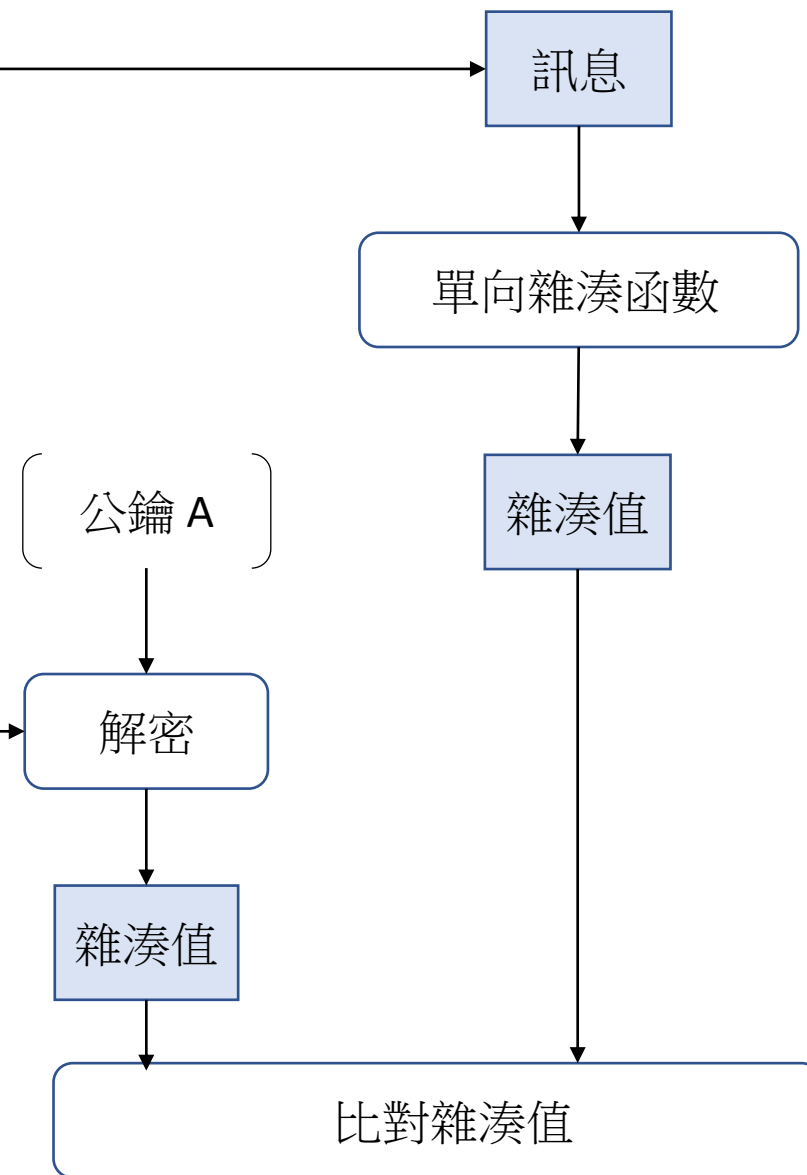
- (1) A 使用單向雜湊函數計算訊息雜湊值
- (2) A 使用自己的私鑰加密雜湊值，並連同訊息一併傳給 B
- (3) B 利用 A 的公鑰來解密收到數位簽章並比對解密得出的雜湊值與收到訊息的雜湊值是否相同

這裡可能有疑問，B 要如何去得到 A 的公鑰同時確保其安全性，這其實是後面所介紹“憑證”的工作，因此這裡先假設 B 通過安全的方式取得 A 的公鑰。

寄件者



收件者



(圖16)

憑證

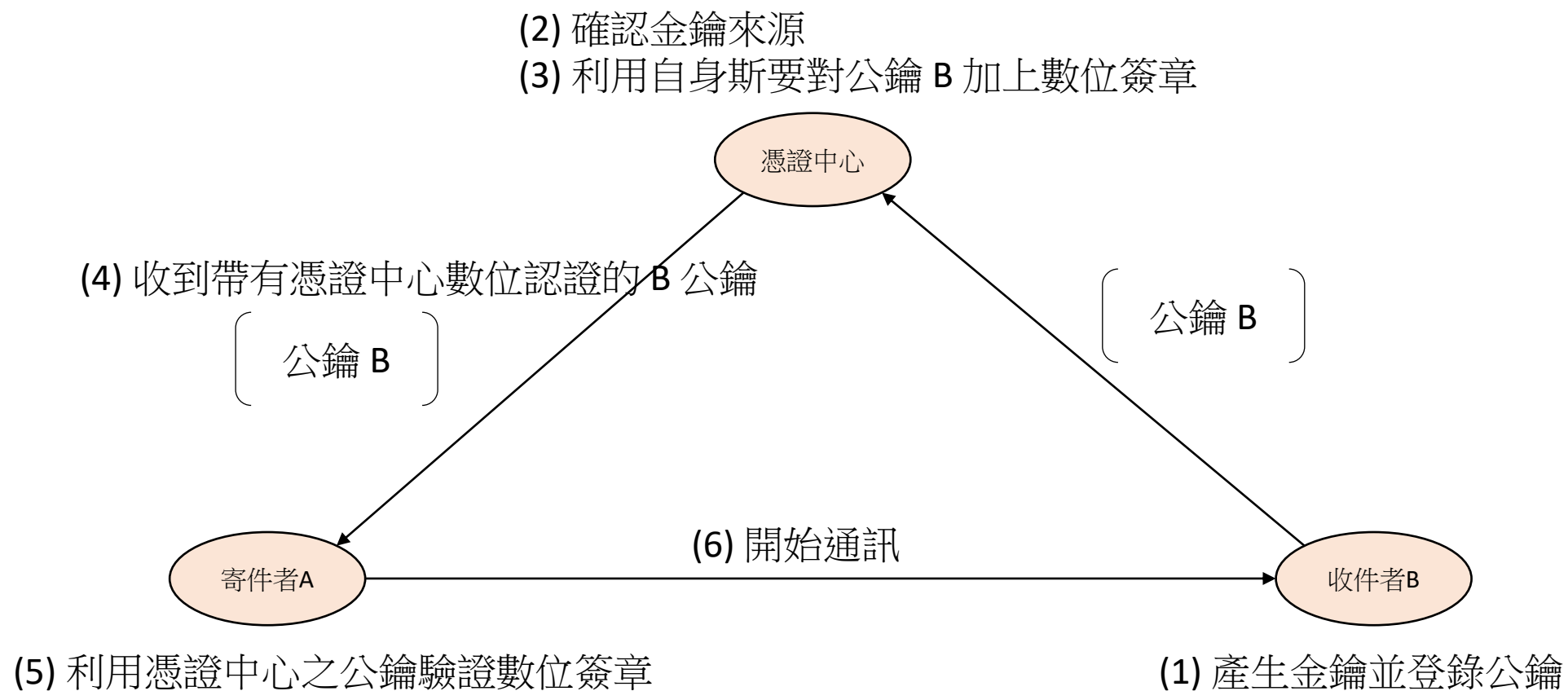
公開金鑰憑證(Public-Key Certification)是一種能確保取得金鑰及其所有者正確來源訊息的功能，公開金鑰憑證會記載姓名、所屬單位、電子郵件等個人資料及他的公開金鑰，再由憑證中心(Certification Authority)來為公開金鑰憑證加上數位簽章。因此，當我們看到公開金鑰憑證時，代表這把公開金鑰的來源及所有者已經被憑證中心所承認了。而憑證中心不一定是公家機關或是跨國大企業才能擔當，也有提供憑證服務的中小企業，甚至是個人。前面講的有點抽象，所以最後總結一句，憑證就是能讓你確定一把金鑰真正使用者的驗證。

憑證的使用步驟:

- (1) B 產生自己的金鑰對並登錄在憑證中心
- (2) 憑證中心確認所登錄金鑰是否屬於 B
- (3) 憑證中心利用自身的私鑰對 B 的公鑰加上數位簽章
- (4) A 收到了帶有憑證中心數位簽章的 B 公鑰

(5) A 利用憑證中心的公鑰驗證數位簽章，並確認 B 的公開金鑰是否正確

(6) 最後A 可以正常的使用 B 的公鑰與 B 通訊



憑證的標準規格

由於憑證是由憑證中心所發行進行驗證，再由使用者依此進行驗證，所以如果憑證的規格補統一且種類亂的話會非常麻煩，因此現在多數憑證都遵守著 ITU 和 ISO 制定的 X.509 規格。X.509 產生憑證並進行交換時的標準規格，支援大部分憑證應用功能。

Email Address	example@example.com
Serial Number	00 B9 2D AB 66 92 68 2A B2
Version	3
Signature Algorithm	SHA-1 with RSA Encryption (1.2.840.113549.1.1.5)
Parameters	none
Not Valid Before	Thursday, September 14, 2017 at 11:39:43 AM Central Daylight Time
Not Valid After	Tuesday, June 30, 2291 at 11:39:43 AM Central Daylight Time
Public Key Info	
Algorithm	RSA Encryption (1.2.840.113549.1.1.1)
Parameters	none
Public Key	256 bytes : BD 94 73 C8 B9 71 93 74 ...
Exponent	65537
Key Size	2048 bits
Key Usage	Verify, Wrap
Signature	512 bytes : 8A BE D9 09 84 FD 1C 42 ...
Extension	Key Usage (2.5.29.15)
Critical	NO
Usage	Digital Signature, Non-Repudiation, Key Encipherment
Extension	(1.2.840.10070.8.1)
Critical	NO
Data	spinnaker-example0%0Aspinnaker-example1

X.509 格式之憑證 ->

公開金鑰基礎架構

儘管已經制定憑證的規格但還是缺少了一個更大範圍的架構，用來解決一些應用上的問題，憑證是由誰製作出來的？私密金鑰外洩後如何撤銷憑證？電腦在交換檔案時該使用哪種形式等等.....，這些都屬於公開金鑰基礎架構 (PKI) 的範疇。

何謂公開金鑰基礎架構

公開金鑰基礎架構(Public-Key infrastructure) 是多種規格及格式的總稱，用來有效運用公開金鑰而制定的。因此，PKI 只是一個總稱而並非單一規格或格式，例如 RSA 所制定的 PKCS 或者是剛剛提到的 X.509 都屬於 PKI，所以，我們在通訊或應用時就會有許多 PKI 可以選擇，但同時我們也較難以掌握 PKI 的全貌。

PKI 的構成元素

PKI 基本上是由以下三個元素所構成：

- (1) 使用者：使用 PKI 的人
- (2) 憑證中心：發行憑證的人

(3) 憑證資料庫：儲存憑證資料的資料庫

使用者

在前面的例子中，**A** 和 **B** 等利用公開金鑰憑證來完成通訊前置的人就可以稱做 **PKI** 的使用者，會執行下列動作：

- 產生金鑰對(有時會交由憑證中心直接產生)
- 向憑證中心登陸金鑰
- 要求憑證中心發行憑證
- 發生特殊情況時向憑證中心撤銷已註冊之公開金鑰
- 加密(或解密)收到的密文
- 對訊息加上(或驗證)數位簽章

憑證中心

簡稱 **CA**，為管理憑證以供驗證的單位，在 **PKI** 中負責下列工作

- 產生金鑰對(有時也會由使用者自行產生)
- 驗證金鑰的登錄者是否為本人
- 產生並發行憑證
- 撤銷憑證

驗證登錄者身分這項工作有時會交由註冊中心(**Registration Authority**，簡稱 **RA**) 來執行，目的是減輕憑證中心工作量使其能專心處理憑證事務。但也由於需要額外對註冊中心進行驗證，構成要素增多，間接導致程序複雜化且能攻擊的點變多

憑證資料庫

憑證資料庫是指儲存憑證資料的單位，提供使用者取得所需的憑證或公開金鑰資料，雖然前面的步驟並未提到憑證資料庫，但只要步驟中有關於憑證存取或讀取的動作都須經過憑證資料庫

接著來詳細看一下憑證中心的工作：

產生金鑰對

在 PKI 中產生金鑰對的單位有兩個，由使用者產生或是直接由憑證中心產生，而如果是由憑證中心產生金鑰對，就必然會執行“將私鑰傳給使用者”的動作，其方法制定在 RFC792(Personal Information Exchange Syntax) 的規格中

登錄憑證

如果金鑰是由使用者產生，就必須向憑證中心請求產生憑證已登錄在憑證資料庫中，申請憑證時的規格是制定在 RFC2986 (Certification Request Syntax Specification) 中，而憑證中心會再確認完登錄者身分後再產生憑證。製作憑證時，會使用憑證中心的私鑰來加上數位簽章，根據 X.509 的格式產生憑證。

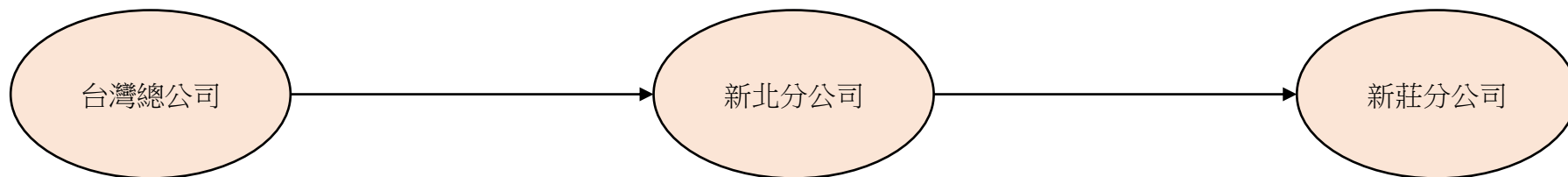
撤銷憑證與 CRL

如果使用者不幸遺失其私鑰或者私鑰被偷，憑證中心必須要撤銷憑證資料庫中對應的公鑰使其無效，除此之外，有時就算私鑰並無變故也需要進行撤銷動作，例如某員工因為離職或其他原因已經喪失了使用該憑證的權限，或者需要修改憑證內的資料都需要重新做一份新憑證並且撤銷舊憑證。撤銷憑證時，憑證中心會做一份“憑證撤銷列表(Certificate Revocation List , 簡稱 CRL)”，CRL 會列出撤銷憑證的序號並對其列表加上數位簽章。

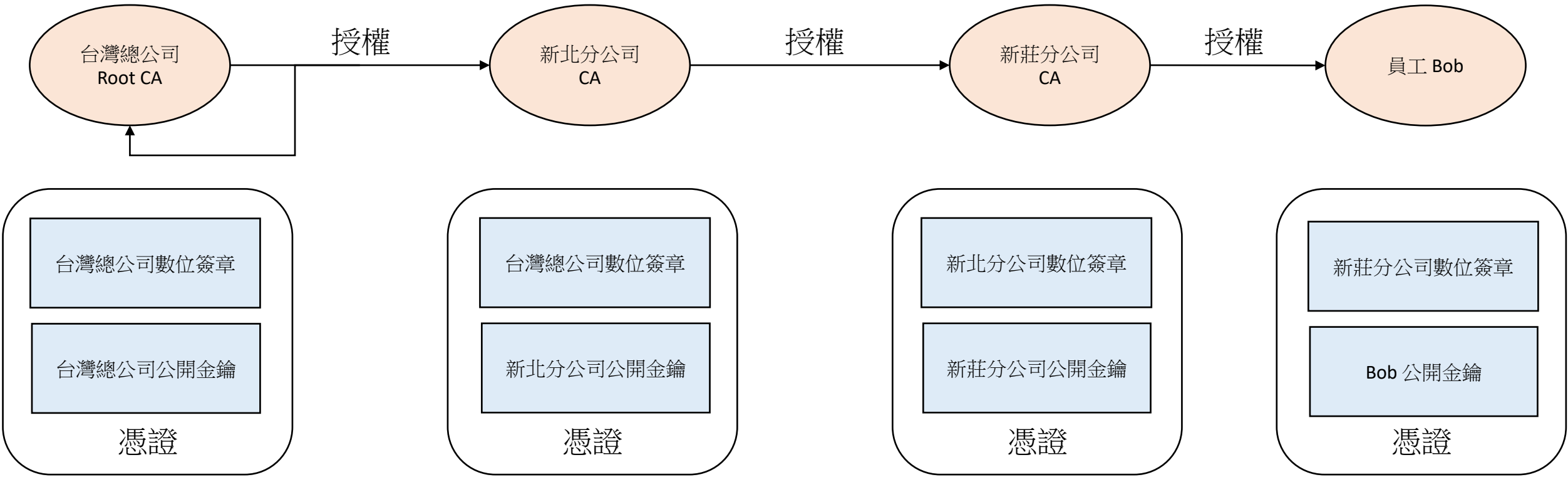
而 PKI 則必須從憑證中心最新發布的 CRL 中來檢查自己驗證的憑證是否已被撤銷。因此在比對最新 CRL 之前就算是在有效期限內且有著正確的數位簽章的憑證也無法確認其有效性，一般來說負責比對 CRL 的工作應該是處理憑證軟體的工作，所以軟體有無定期更新 CRL 是維持憑證安全性的一環

憑證的階層結構

接著要來說明如何判斷憑證中心所提供用來驗證數位簽章的公鑰本身是否正確，針對憑證中心本身的公開金鑰，可以去登錄其他憑證中心來驗證其正確性，相當於憑證中心本身變成了新的 PKI 使用者，亦即製作憑證中心的公開金鑰憑證。這種以憑證中心驗證另一憑證中心公開金鑰得動作可以重複執行並延伸，常見的應用情形為具有階層關係的憑證中心。假如有一間大公司具有許多分店且有各自的憑證中心，並有著下列階層關係



假設某員工 **Bob** 是新莊分公司的員工，而新莊分公司的憑證公司可以輕易地驗證 **Bob** 的身分並對其公開金鑰發行憑證。接著將規模拉到公司單位，新北分公司對新莊分公司的憑證中心公開金鑰發行憑證，台灣總公司再對新北分公司的憑證中心公開金鑰發行憑證，這重複執行的發行憑證動作最後停在了台灣總公司 (也就是該憑證中心的最上層)，而這個最上層的憑證中心也稱為“根憑證(**root CA**)”，而根憑證由於沒有能為它授權的憑證中心，所以只能為自己的公開金鑰加上自己的數位簽章，稱作自我簽章。



假設現在某一 PKI 使用者 Alice 想要驗證 Bob 的數位簽章，Alice 需要依下列步驟來取得 Bob 的公開金鑰：

(1) 取得根憑證的公開金鑰

在取得 Bob 的公開金鑰之前，必須經過各上層驗證才可以完成最後公開金鑰的驗證，一步步往上推之後我們能發現一切的驗證源頭為根憑證的公開金鑰，取得到根憑證的公開金鑰後再根據皆能慢慢向下驗證

(2) 依序向下驗證

取得完根憑證 (台灣總公司) 的公開金鑰後，我們能向下驗證新北分公司的憑證，在利用其公開金鑰驗證新莊分公司的憑證，最後就能用新莊分公司的公開金鑰認證 Bob 的憑證並取得其公開金鑰

(3) 完成並開始通訊

取得到 Bob 的公開金鑰後就可以與 Bob 開始通訊了。當然，以上述這些繁瑣的步驟並非由人工來執行，而是由電子郵件或瀏覽器軟體代為操作

各種 PKI

我們常以為 PKI 架構中的 CA 或 root CA 只有具權威性或具有一定規模的企業才能作，但其實不然，因為運作核心其實不複雜，就是對公開金鑰加上數位簽章，所以除了讓具有權威性的企業當個商業來往的 CA，在一般的企業、政府機關或是自治團體等情況都可以依照相關的協定架構出各種用途的 PKI，除此之外，PKI 中的階層分布也有不同的用途，總之並沒有一定的制式做法

對憑證的攻擊

- 在公開金鑰登錄前攻擊

由於對登錄完的公開金鑰攻擊會很困難，因此會在金鑰登錄前就做攻擊，攻擊者可能會攔截登錄信息並將公開金鑰換成自己的，要防範此類攻擊除了加強身分驗證之外，也可以在登錄時加入數位簽章動作等

- 登錄類似現有使用者

這是一種類似混淆使用者的方式，假如攻擊者要假冒 B 來與 A 進行通訊，而 B 在 CA 註冊的使用者為 Bob，攻擊者就去相同的 CA 註冊一個名為 BOB 的使用者，再將自己的憑證資訊傳給 A，如果 A 沒有與 B 加以確認，很可能就將訊息誤傳給了攻擊者。而如果要防範此種攻擊，CA 必須要在驗證身分的方面加強管理，而使用者在使用憑證時也要再三確認憑證資訊是否與通訊對象相符。

- **竊取憑證中心的私密金鑰**

非常直觀且暴力，此方式直接竊取 CA 的私鑰來製作憑證，而且效力等同於原 CA 所發行的 (因為是同把私鑰)，所以一個 CA 能否保護好自己的私鑰及安全資料與這個 CA 是否能被信任有著直接的關係，所以防範的方式唯有 CA 方有著一定的資安意識並積極做好防護措施。

- **攻擊者假冒憑證中心**

這是一個聽起來很大膽的攻擊，由於經營一間憑證中心並不需要登記或是實體空間，只需要用經營憑證中心的專用軟體，誰都可以成立憑證中心，因此，在這類攻擊中攻擊者直接成立了一個憑證中心，假設攻擊者還想了辦法取得了其他憑證中心的憑證，那他就可以將自己的公開金鑰當作 Bob 的公鑰並予以憑證發給 A，而 A 有可能就會相信此憑證而將訊息用攻擊者的公鑰加密。而要防範此攻擊法，使用者一定要再三確認取得訊息的憑證中心是否值得信任且有取得其他憑證中心的憑證。

- **CRL 時間差攻擊**

接著來介紹 CRL 的時間差攻擊，我們以下列兩種情況來說明如何利用時間差來攻擊：

- (1) **竊取私鑰使 CRL 來不及更新**

這種情況發生在使用者 **B** 的私鑰被攻擊者直接盜取，並直接用 **B** 的私鑰發了一則附有數位簽章的訊息要求 **A** 轉錢給自己的帳戶，而 **A** 在驗證完訊息的數位簽章也不假思索地轉錢給攻擊者，過了一段時間，**B** 終於發現自己的私鑰被竊且拿來做未經自己授權的數位簽章，儘管第一時間要求 **CA** 撤銷憑證，但 **A** 的錢早已轉了出去。其實此攻擊最主要的問題是私鑰被盜取 (如何盜取這裡不做討論可以參考前面公開金鑰系統)，所以我們只能盡量快速地通報 **CA** 且撤銷有問題的憑證來“減少”損失

(2) CRL 時間差導致否認行為

在這個情況中攻擊者不是別人，正是使用者 **B**，具體情況為 **B** 先利用自己的私鑰製作一份帶有數位簽章的訊息告知 **A** 往自己的秘密帳戶轉錢，在確認秘密帳戶收到錢後馬上告知 **CA** 自己的私鑰被盜並要求撤銷憑證，在密碼學的角度中 **B** 成功的否認掉了轉錢訊息由自己所發，而此種攻擊很難用公開金鑰或憑證技術發現 **Bob** 的犯罪行為，而此時我們也只能請警方展開犯罪調查，也許能從 **B** 電腦中的活動紀錄與其他證據去發現犯人就是 **B**

所以到底誰可以被信任？

經過上述對憑證的簡介後，我們可以發現其實很多問題點都是我們是否能“信任”我們使用的 **CA**，而這我覺得我們可以先以銀行為例，我們可以透過網路、口碑、新聞或是公司行號之類的去評斷是否能將錢託付其保管；**CA** 也是一樣，我們會通過各種能信賴的消息來源來確認我們所使用的 **CA** 是否安全。

金鑰

在認識了許多不同組合的應用後，回來補充說明一下關於前面沒有講清楚的金鑰知識

- 金鑰的價值等同於明文資訊

如果一份資訊靠著金鑰加密來維持其機密性，那麼加密的那把金鑰價值等價於機密資訊的內容，因為只要取得那把加密的金鑰誰都可以去得知機密訊息的內容，因此在加密完後如何保護金鑰直接關聯著機密資訊的安全

- 金鑰用途

前面介紹的應用中，我們知道了金鑰的公用不只用於加密訊息，也可以用來加密其他金鑰 (混和型密碼)、驗證身分 (數位簽章)、檢查完整性 (訊息驗證碼) 等等.....，因此要注意金鑰在每個情況下的作用到底是什麼

- 演算法公開

很多人對安全性有一個謬思是只要不公開加密或安全演算法，就會比公開還要安全，因為攻擊者不知道我的整體機制，但這其實是錯誤的，一個安全的演算法是需要公開並讓大家來考驗此演算法的安全性，相反地，如果演算法被隱藏而無法讓大眾檢驗，那麼此演算法就較有可能會有漏洞成為攻擊目標。

產生金鑰

(1) 利用亂數產生器

產生金鑰最好的方式是利用專門的亂數產生器來產生對應長度的金鑰，這樣就能完全屏除個人的意念影響金鑰進而造成被破解，但同時亂數產生器的重要性也隨之增加，所以務必要使用專門設計給密碼用的亂數產生器，這樣能有效避免亂數被預測進而造成危險

(2) 利用個人密碼產生金鑰

有時我們也能透過人類能記住的個人密碼來產生金鑰，簡單來說會將個人密碼當作金鑰並經過一系列的運算後產生出金鑰，隨後會詳細介紹

配送金鑰

如何安全的去配送共用金鑰給對方是很重要的一環，解決辦法在前面的公開金鑰系統有提到，因此隨後也會講解前面為詳細介紹的 Diffie Hellman 金鑰交換

刪除金鑰

在使用者 A 和 B 通訊完之後共用金鑰對兩人來說已無價值，但卻有可能變成攻擊者竊取通訊資料的漏洞，所以使用完的金鑰必須鑰確實地刪除以絕後患

如何刪除

假如金鑰是儲存在電腦上的檔案，光刪除檔案是算不上真正刪除金鑰的，因為現今市面上已有許多技術能將刪除或受損的檔案復原，所以必須連軌跡都一並刪除。換句話說，如果需要徹底刪除金鑰，除了軟體的設計外，硬體也必須特殊處理。

金鑰遺失

這就麻煩了，儘管證實了金鑰遺失的事實並提出撤銷憑證等相關處理，先前有用到此金鑰的動作或是權限都會失效，還有可能造成資產的遺失，例如之前就傳出有人在比特幣大漲之前就有屯幣，但當比特幣真的大漲時卻忘記當時的密碼，只能眼睜睜的看著卻拿不到 (比特幣的特色即為去中心化，意指沒有中介機構能幫他找回密碼)

Diffie Hellman

這種演算法可以讓兩人在不安全的通訊通道中安全的得出只有兩人知道共同數值 (或資料)，原理其實跟 RSA 有點像，運用離散對數來在不安全的通道中產生相同的數值

Diffie Hellman 交換金鑰程序

假設 Amy 和 Bob 想要在不安全 (可能被監聽) 的通道中產生一共用金鑰用於之後通訊的加密，這時就是使用 Diffie Hellman 的時機

(1) Amy 傳送兩質數 P 與 G 給 B

P 必須是一個非常大的質數，而 G 則為依附 P 所決定的生成元 (後面會解釋，這裡只要先知道 G 會介在 $1 \sim P-2$ 之間)，而且 P 和 G 就算被監聽也沒關係

(2) Amy 準備一亂數 A

接著 Amy 需要準備一介於 1 到 $P-2$ 的亂數 A ，這個亂數 A 只能讓 Amy 自己知道，不能讓其他人 (包含 Bob) 知道

(3) Bob 也要準備亂數 B

跟 A 一樣，Bob 也需要準備一個只有自己知道的亂數 B (介於 $1 \sim P-2$)

(4) Amy 將 $G^A \bmod P$ 的值傳送給 Bob

這個值就算被竊聽也沒差

(5) Bob 將 $G^B \bmod P$ 的值傳送給 Amy

這個值就算被竊聽也沒差

(6) Amy 將收到的值再乘以 A 次方，之後 $\bmod P$

所以會得到 $G^{BA} \bmod P$

(7) Bob 將收到的值也乘以 B 次方，之後 $\bmod P$

所以會得到 $G^{AB} \bmod P$

最後兩人成功得到了相同值的共同金鑰 (圖17)

攻擊者是否能算出兩人的共同金鑰

非常困難，因為共同金鑰依靠著離散對數難題保護著，攻擊者能竊聽到以下兩個值

$G^B \bmod P$

$G^A \bmod P$

但是攻擊者只知道 **G** 和 **P** 的值的話，無法推出 **A** 和 **B** 兩值，因為至今依舊沒有一有效的演算法能從 $G^A \bmod P$ 靠著 **G** 和 **P** 推出 **A**，這就是前面講的離散對數難題

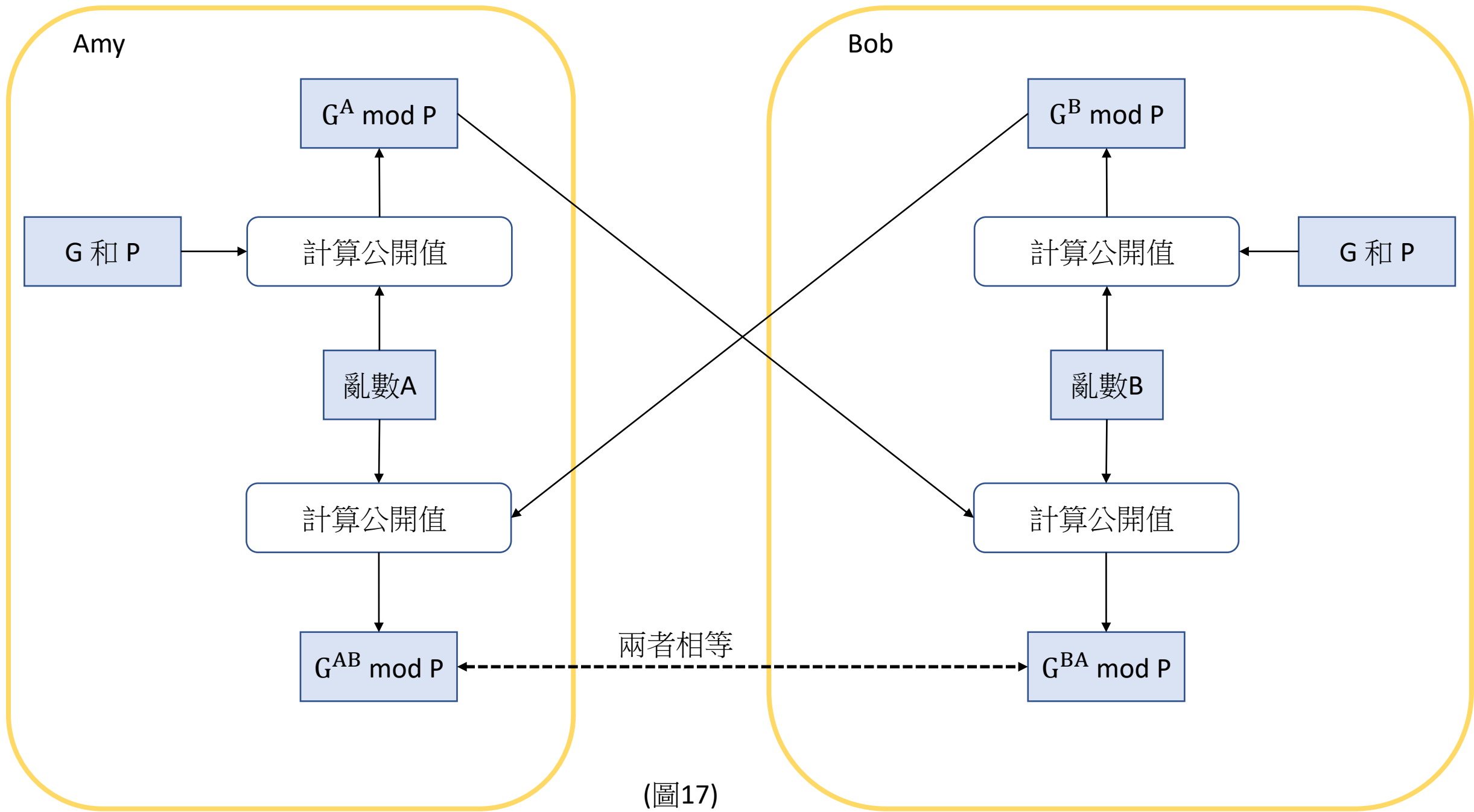
生成元是什麼

請先觀察右表 (假設 **P = 13**)

表 1.1: $G^A \bmod P$ 表 ($P = 13$)

$G \backslash A$	1	2	3	4	5	6	7	8	9	10	11	12
0	0	0	0	0	0	0	0	0	0	0	0	0
1	1	1	1	1	1	1	1	1	1	1	1	1
2	2	4	8	3	6	12	11	9	5	10	7	1
3	3	9	1	3	9	1	3	9	1	3	9	1
4	4	3	12	9	10	1	4	3	12	9	10	1
5	5	12	8	1	5	12	8	1	5	12	8	1
6	6	10	8	9	2	12	7	3	5	4	11	1
7	7	10	5	9	11	12	6	3	8	4	2	1
8	8	12	5	1	8	12	5	1	8	12	5	1
9	9	3	1	9	3	1	9	3	1	9	3	1
10	10	9	12	3	4	1	10	9	12	3	4	1
11	11	4	5	3	7	12	2	9	8	10	6	1
12	12	1	12	1	12	1	12	1	12	1	12	1

我們可以特別注意到當 $G = 2、6、7$ 和 11 時，當 A 次方在 $1 \sim 12$ 的各元素中，剛好出現了 $\mathbb{Z}/13\mathbb{Z}$ 的所有元素且不重複，而具有上述特性的次方 A 即可稱為 P 的生成元。換句話說， P 的生成元經過次方計算再 $\text{mod } P$ 之後，會與 $1 \sim P-1$ 的值成一對一的對應關係，因此我們能在 $1 \sim P-2$ 隨機選一個 A 來產生 $G^A \text{ mod } P$ (根據歐拉定理， $G^{(P-1)} \text{ mod } P \equiv G^{\varphi(P)} \text{ mod } P \equiv 1 \text{ mod } P$ ，所以我們不能由 $P-1$ 當作 A)



個人密碼的加密演算法

最後我們來介紹一下根據個人密碼產生金鑰的演算法 PBE (Password Based Encryption)，顧名思義就是以個人密碼來產生加密用的金鑰，加密與解密都需要相同的個人密碼。而 PBE 的特點在於我們可以用人腦能記住的密碼長度來保存大長度的加密金鑰，如此就可以不用煩惱金鑰要如何安全保存的問題，PBE 的規格其實非常多元，所以等等的介紹會專注在大致的架構

加密步驟

PBE 大致的加密步驟可以分成下列三階段

- (1) 產生 KEK (金鑰加密金鑰)
- (2) 產生 CEK (對話金鑰) 並加密
- (3) 加密訊息

詳細步驟如下

(1) 產生 KEK

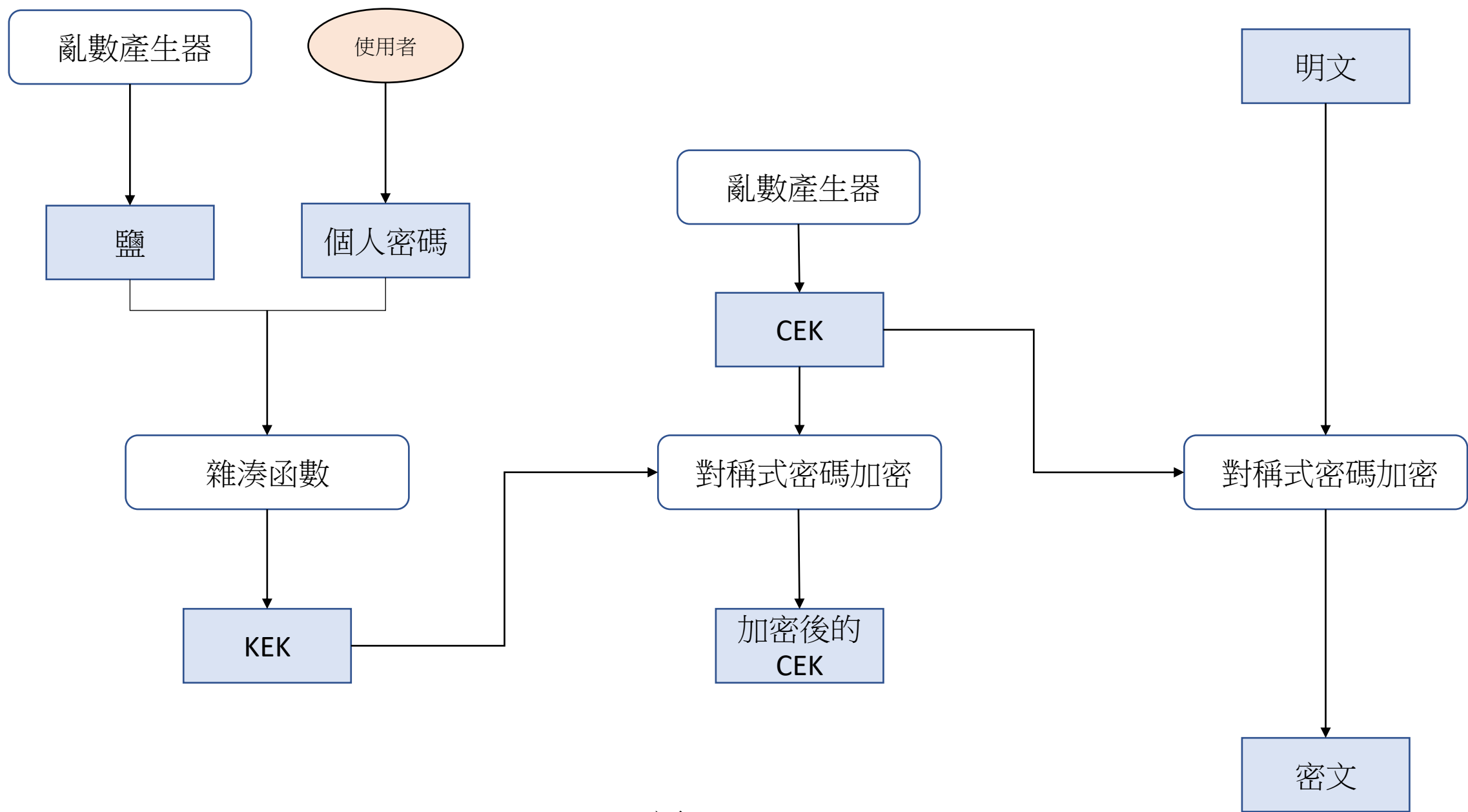
首先，利用亂數產生器產生稱作“鹽 (salt)”的亂數，再將使用者的個人密碼與鹽依序加入雜湊演算法中，產生的雜湊值即為等等鑰加密對話金鑰的金鑰。而鹽的功用主要是打亂個人密碼的雜湊值，以防遭受字典攻擊或彩虹表攻擊

(2) 產生對話金鑰並加密

接著再用亂數產生器產生出對話金鑰，對話金鑰會成為加密明文的金鑰 (CEK)，最後使用剛才的 KEK 加密對話金鑰，並與鹽一同存在安全的場所，之後 KEK 就可以丟棄了，因為只要有鹽和個人密碼就可以創造出 KEK 並得出 CEK (但是鹽和加密後的 CEK 需要先儲存在安全的地方)

(3) 加密訊息

最後使用 (2) 所產生的 CEK 來加密訊息即可



(圖18)

解密步驟 (圖19) :

(1) 還原 KEK

首先我們必須先將事先儲存的鹽與個人密碼進行雜湊得出 KEK

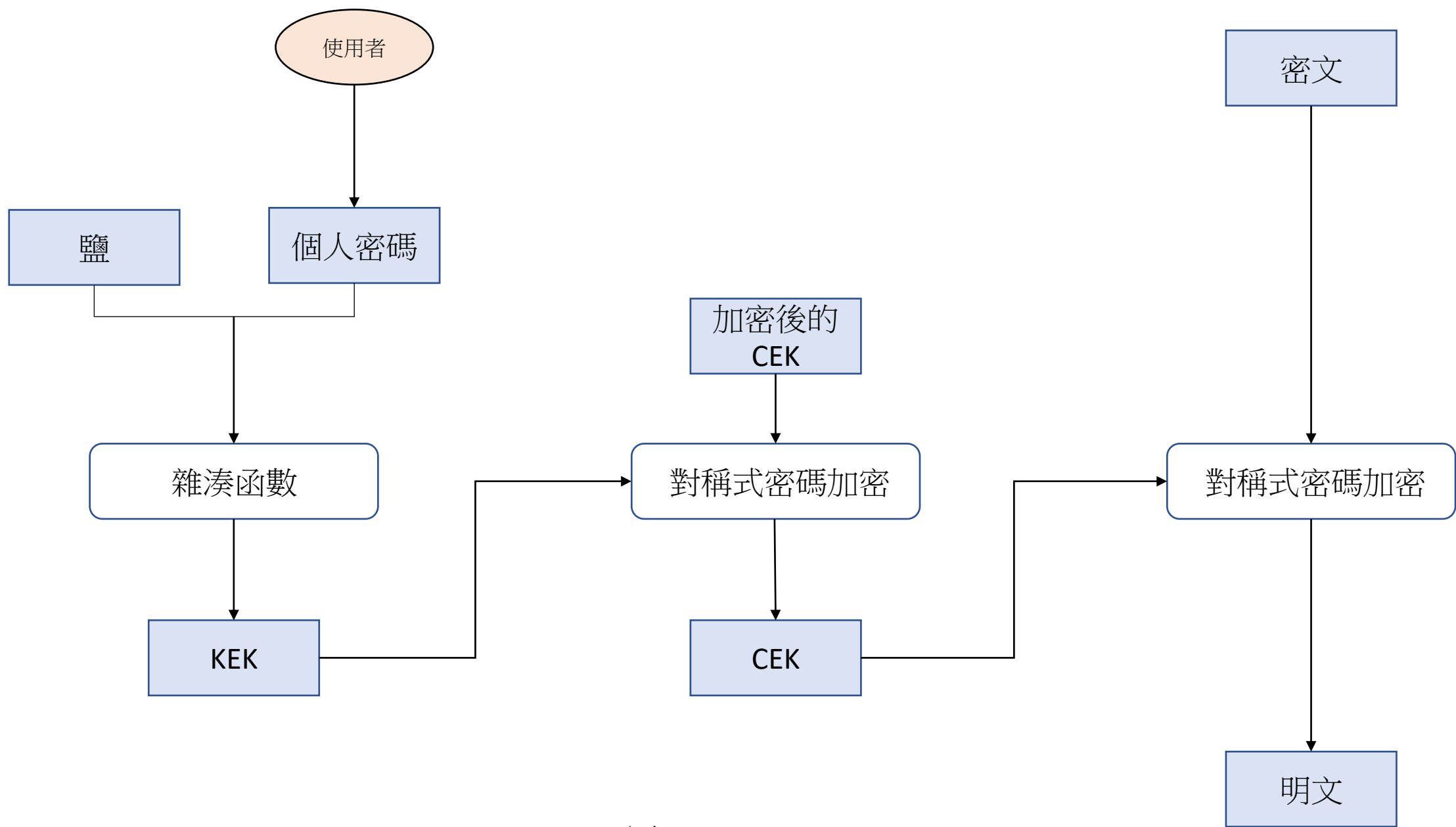
(2) 解密對話金鑰

接下來取出事先儲存價密後的 CEK 值，並用剛剛還原出的 KEK 進行解密得出 CEK

(3) 解密訊息

有了 CEK 我們就能將密文解密成明文了

在解密的過程中，完全沒有亂數產生器的出現及使用



(圖19)